

The background is a dark, textured surface with a complex network of thin, yellow lines connecting various blue, three-dimensional cubes. The cubes are of different sizes and are scattered across the frame, creating a sense of depth and connectivity. The overall aesthetic is technical and digital.

Digital Logic Design

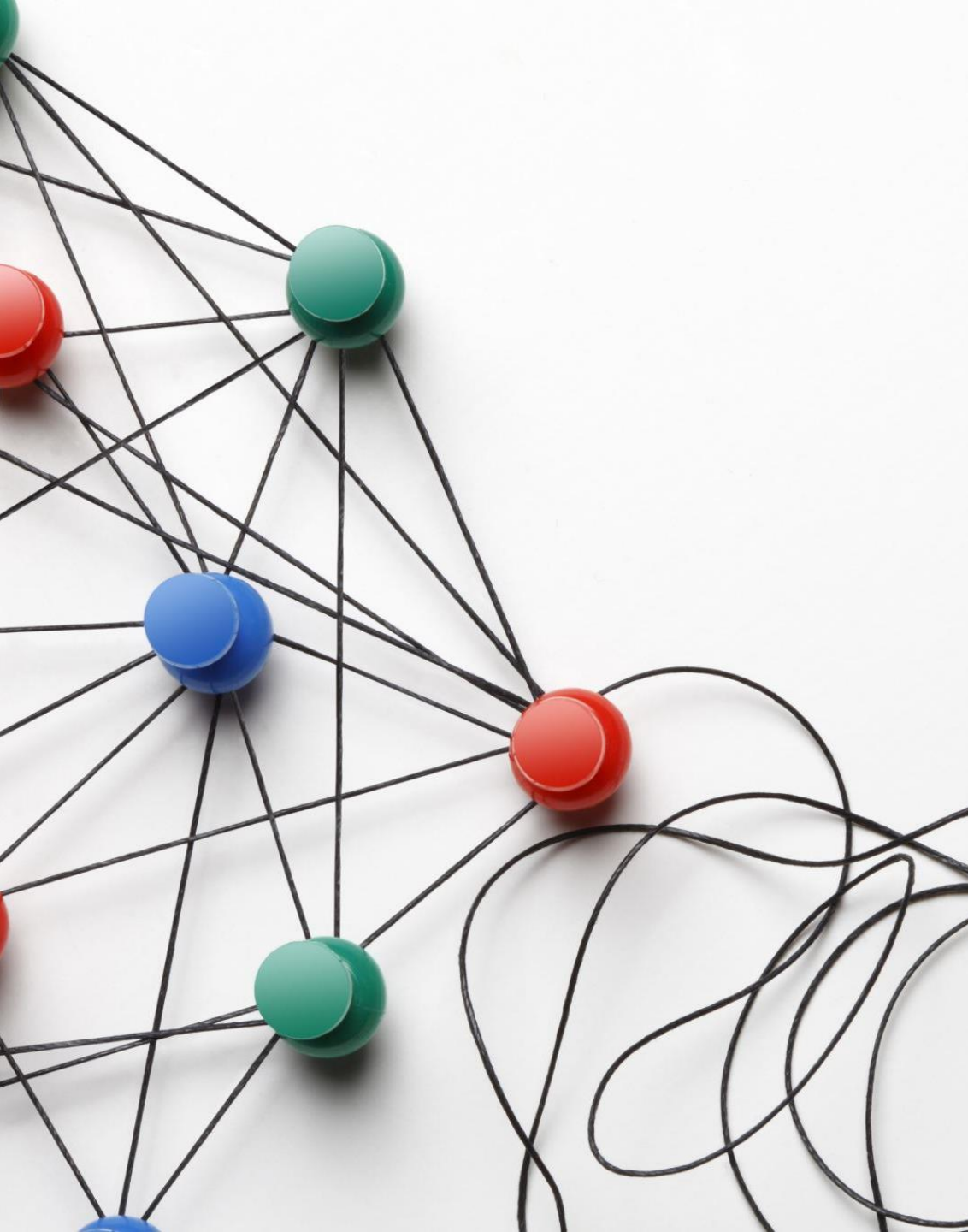
Akila Subasinghe

Department of Computer Science

University of Birmingham, Dubai

Topic 2 Outline

- Basic logic functions, expressions & truth tables
- Logic gates, circuits, & finding the truth table
- Simplifying logic expressions using Boolean algebra and Karnaugh maps
- Common combinatorial, i.e., no memory, logic circuits
- Sequential, i.e., with memory, logic circuits



Session Topics

- Explain the purpose of logic circuits and binary representation.
- Construct and interpret truth tables for logic functions.
- Identify and describe standard logic gates.
- Represent Boolean expressions using circuits and vice versa.

Why Learn Digital Logic?

Digital logic is fundamental to:

- Computers and Microprocessors – ALU, control units.
- Embedded Systems – Elevators, traffic lights, washing machines.
- Decision-Making Circuits – Comparators, voting systems.
- Data Transmission and Error Checking.

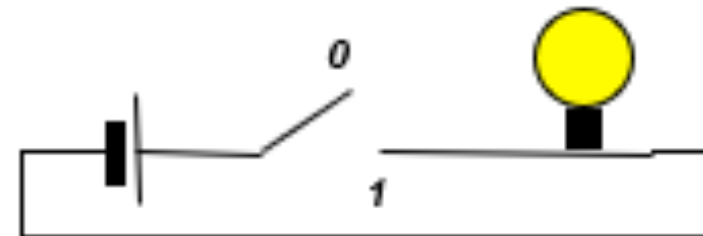
Introduction to Logic Gates & Truth Tables

Recall...

- Digital signals operate using two voltage levels:

Logic State	Voltage Level	Representation
LOW	0v	0
HIGH	+5v / 3.3v	1

- Binary systems use only 0 and 1, which makes circuits noise resistant.



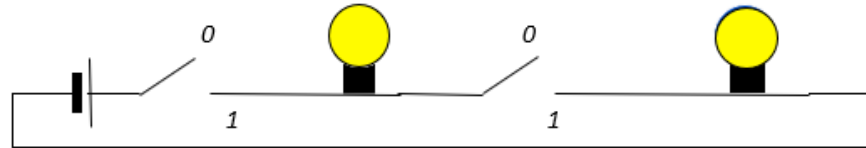
What is a Truth Table?

- Outcomes of these digital systems will be represented on a “Truth Table”
- A truth table lists all possible input combinations and their corresponding output values.
 - Used to define the behavior of a logic function.
 - For n inputs $\rightarrow 2^n$ possible input combinations.

A	B	Y
0	0	?
0	1	?
1	0	?
1	1	?

Switches in Series (AND Gate)

- Two switches can be used in series



- The bulb is on if switch A AND switch B are both on
 - $f = A \cdot B$
 - $\cdot$ is used to denote AND in Boolean algebra (also $\wedge$)

Switches in Series (AND Gate) Cont

- The truth table for two switches in series $f = A \cdot B$ is shown below

A	B	F = A.B
0	0	0
0	1	0
1	0	0
1	1	1

- Interpretation: Output is **1 only when both inputs are 1**.



Switches in Series (AND Gate) Cont

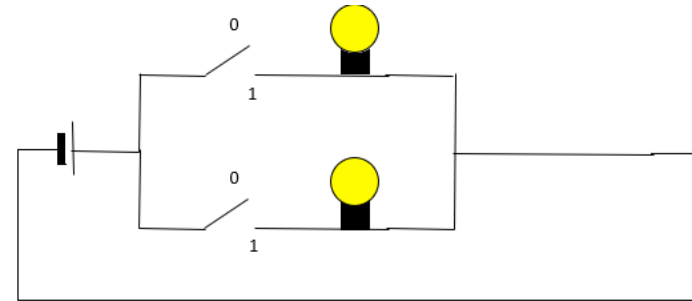
- Q. Find the output for the following two 8bit inputs through AND gate.



A	B	F = A.B
0	0	0
0	1	0
1	0	0
1	1	1

Switches in Parallel (OR Gate)

- We can also consider two switches in parallel



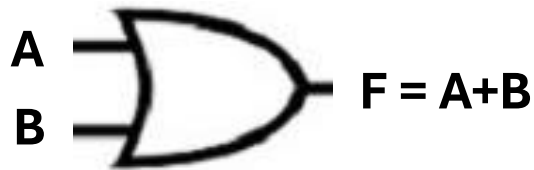
- The bulb is on if switch A OR switch B are on
 - $f = A + B$
 - + is used to denote OR in Boolean algebra (also V)

Switches in Parallel (OR Gate) Cont

- The truth table for two switches in parallel $f = A + B$ is shown below

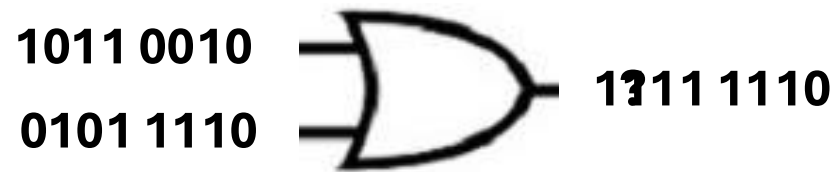
A	B	F = A+B
0	0	0
0	1	1
1	0	1
1	1	1

- Interpretation: Output is **1 only when Any inputs are 1.**



Switches in Parallel (OR Gate) Cont

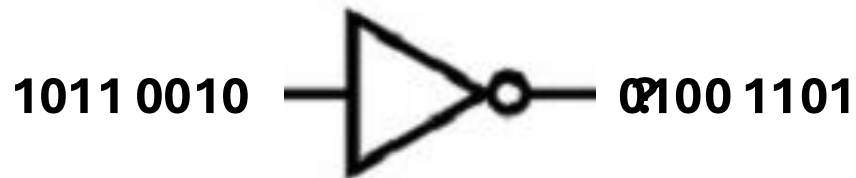
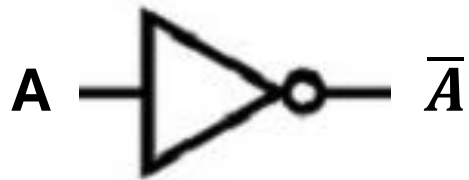
- Q. Find the output for the following two 8-bit inputs through OR gate.



A	B	F = A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

- What if we want to invert a signal ($1 \rightarrow 0$ or $0 \rightarrow 1$)?
- A NOT gate takes the inverts of any variable/signal.
- In notation, we use $\bar{A}$ (A bar) to represent the inverted A

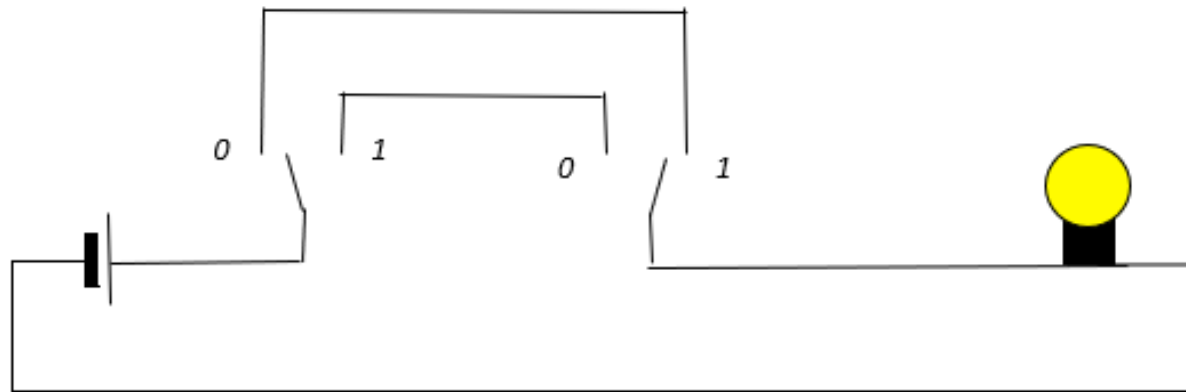


A	$F = \bar{A}$
0	1
1	0

Exclusive OR (EX-OR)(XOR) Gate

- A unique logic gate...
- The bulb is on when only one switch is on

$$F = A \oplus B$$



A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive OR (EX-OR)(XOR) Gate

- A unique logic gate...
- The bulb is on when only one switch is on

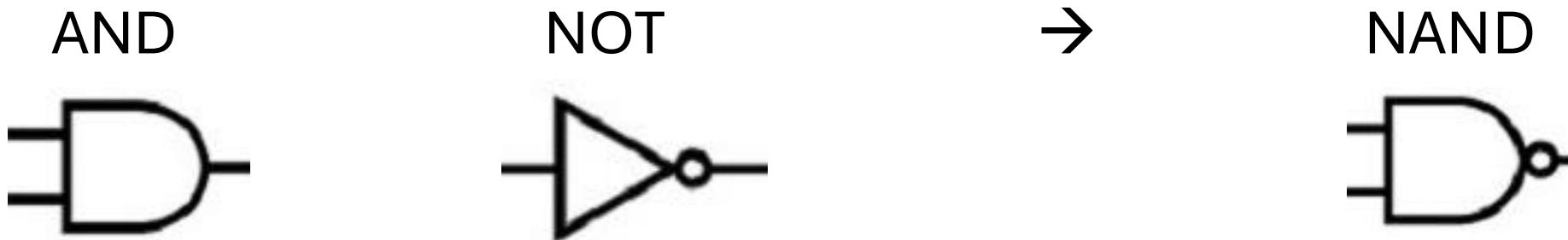
$$F = A \oplus B$$



A	B	F = A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

What adding a NOT does

- Adding a NOT gate in front of AND, OR and XOR gates give us a different gate, which has the opposite behavior of the original gate.
- Note the “o” symbol in the diagram.
- We add the letter “N” to the gate name.
- Example



NAND Gate

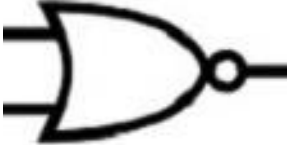
A	B	$F = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0



NOR Gate

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

1011 0010
0101 1110

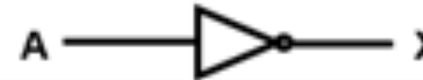


0000 0001

XNOR Gate

A	B	$F = \overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1



Name	Symbol	Function	Truth Table															
AND		$X = A \cdot B$ or $X = AB$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	0	1	0	0	1	1	1
A	B	X																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$X = A + B$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	1
A	B	X																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
I		$X = A'$	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0									
A	X																	
0	1																	
1	0																	
NAND		$X = (AB)'$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	1	0	1	1	1	0	1	1	1	0
A	B	X																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$X = (A + B)'$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	0
A	B	X																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR Exclusive OR		$X = A \oplus B$ or $X = A'B + AB'$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	0
A	B	X																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
XNOR Exclusive NOR or Equivalence		$X = (A \oplus B)'$ or $X = A'B' + AB$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	1
A	B	X																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Logic Circuit Technologies & Universal Gates

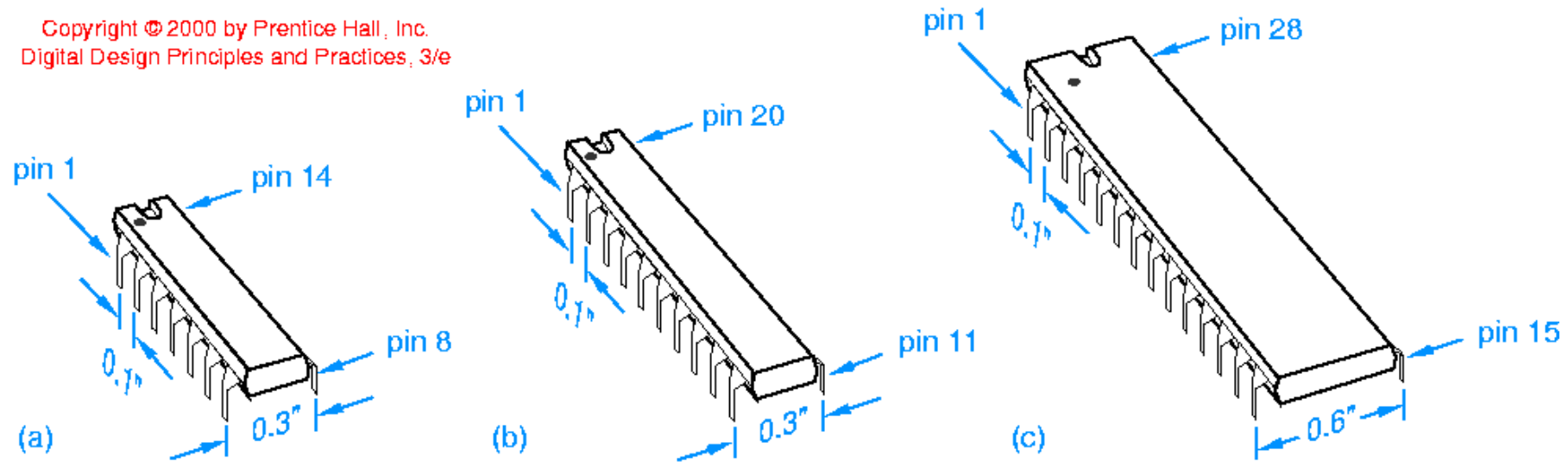
Generations of Computer Systems

Generation	Approximate Dates	Technology	Typical Speed (operations per second)
1	1946–1957	Vacuum tube	40,000
2	1958–1964	Transistor	200,000
3	1965–1971	Small and medium scale integration	1,000,000
4	1972–1977	Large scale integration	10,000,000
5	1978–1991	Very large scale integration	100,000,000
6	1991-	Ultra large scale integration	1,000,000,000

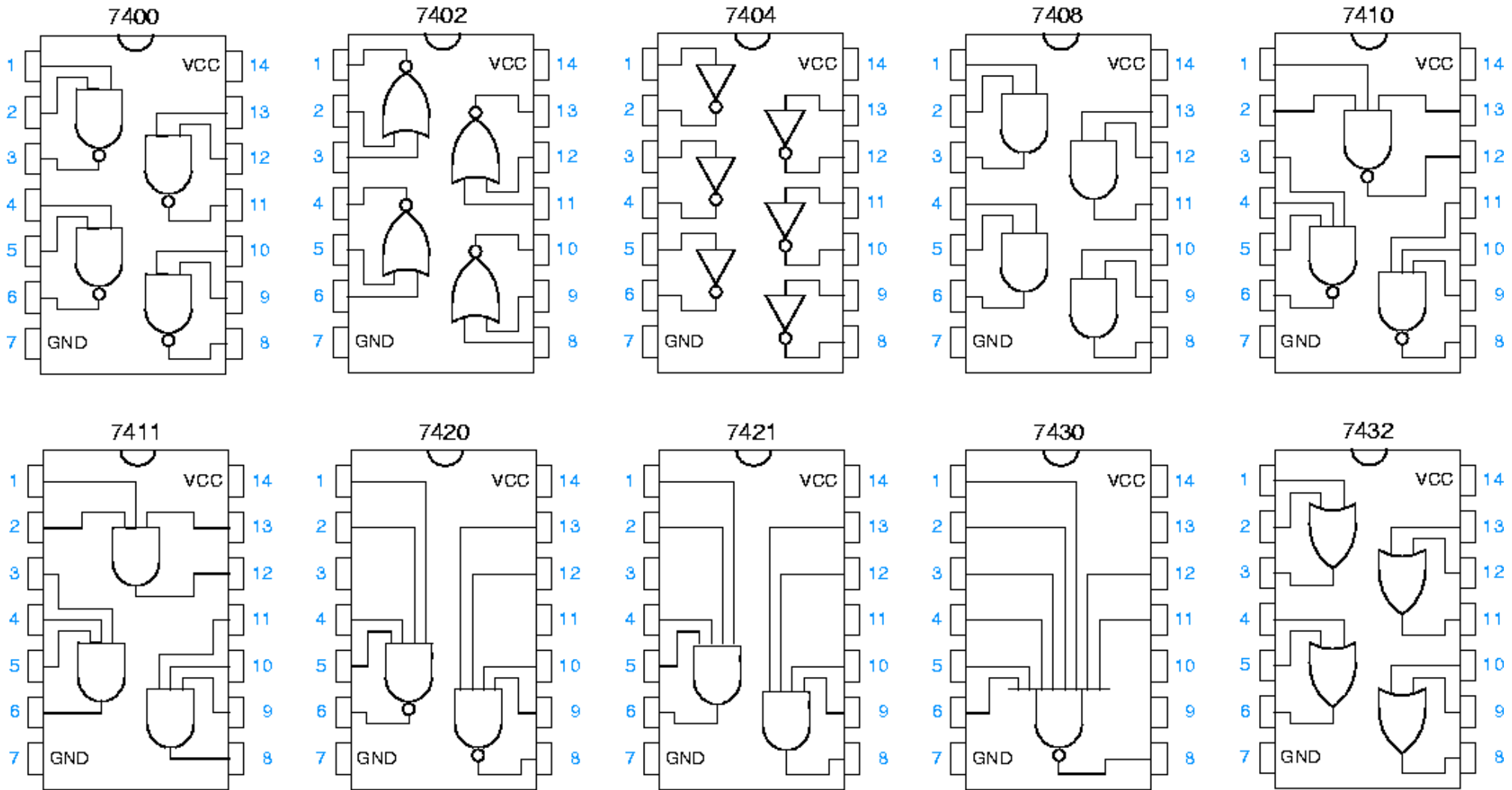
Reproduced from: W.Stallings, “Computer Organization and Architecture”, 9th Edition, Pearson, 2013

Physical Packaging of Logic Gates

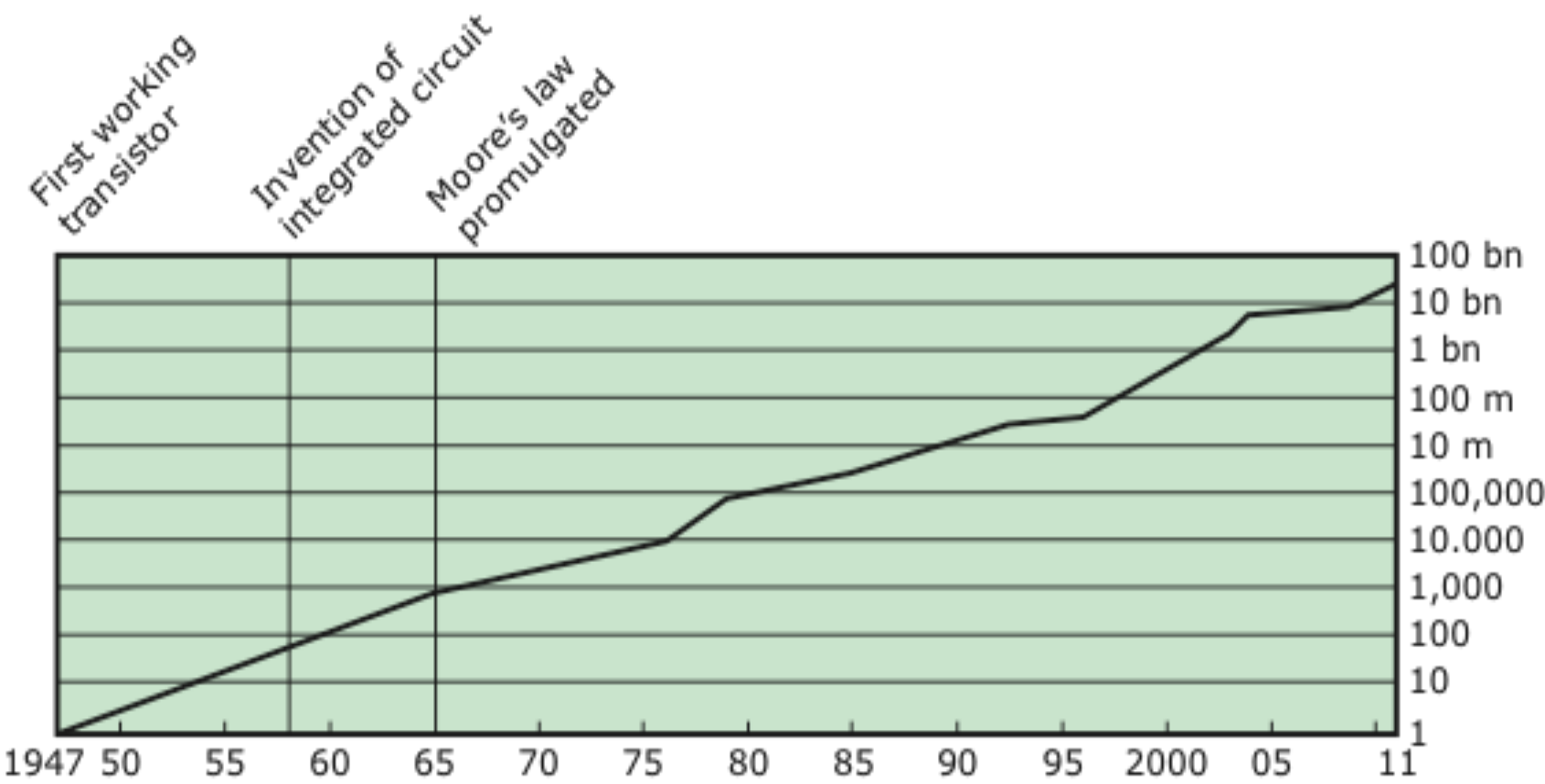
Copyright © 2000 by Prentice Hall, Inc.
Digital Design Principles and Practices, 3/e



Physical Packaging of Logic Gates



Moore's Law



Reproduced from: W.Stallings, "Computer Organization and Architecture", 9th Edition, Pearson, 2013

Universal Gates

- NAND and NOR are called universal gates.
- Any Boolean function can be implemented using only NANDs or only NORs.
- They reduce manufacturing cost and are common in IC design.

NAND

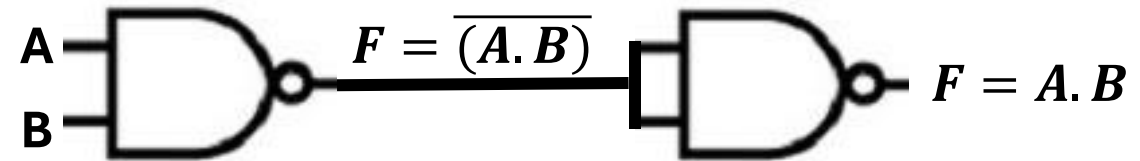
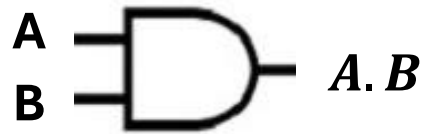


NOR



Implementing AND using NAND

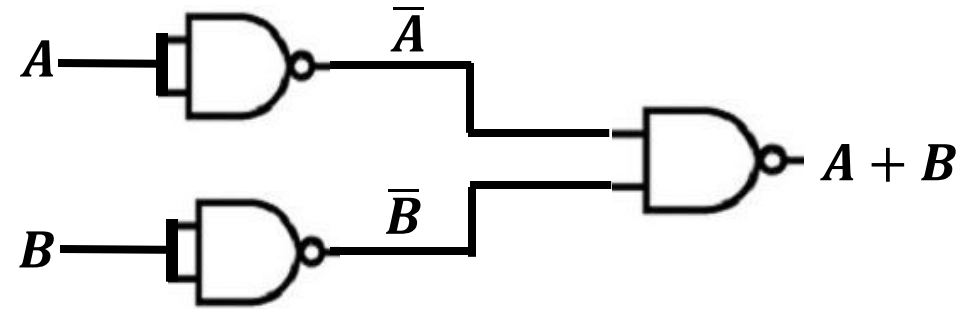
- We can check this with a truth table



A	B	$\overline{(A.B)}$	$A.B$
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Implementing OR using NAND

- We will learn this derivation after studying a bit more on Boolean Logic



A	B	$\overline{(A)}$	$\overline{(B)}$	$A + B$
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

Truth Table to Logic Expression

- This conversion can be done using two methods.

Term	Meaning	Output Relation
Minterm	A product (AND) term where each variable appears once in either true or complemented form.	Represents rows where $F = 1$
Maxterm	A sum (OR) term where each variable appears once in either true or complemented form.	Represents rows where $F = 1$

Truth Table to Logic Expression

- Consider the following truth table, and use “Minterms” to derive the expression.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Truth Table to Logic Expression

- Consider the following truth table, and use “Minterms” to derive the expression.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Truth Table to Logic Expression

- Consider the following truth table, and use “Minterms” to derive the expression.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$F = \bar{A}.\bar{B}.C$$

Truth Table to Logic Expression

- Consider the following truth table, and use “Minterms” to derive the expression.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$F = \bar{A}.\bar{B}.C + \bar{A}.B.C$$

Truth Table to Logic Expression

- Consider the following truth table, and use “Minterms” to derive the expression.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$F = \bar{A}.\bar{B}.C + \bar{A}.B.C + A.\bar{B}.\bar{C}$$

Truth Table to Logic Expression

- Consider the following truth table, and use “Minterms” to derive the expression.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$F = \bar{A}.\bar{B}.C + \bar{A}.B.C + A.\bar{B}.\bar{C} + A.B.\bar{C}$$

Circuit diagram – Logic Expression – Truth Table

- Given one of the above, you should easily formulate the other two.
- The method used in the previous slide for deriving the logical expression may not be the most elegant one.
- We will learn how to simplify them after a couple of exercise questions.

Circuit diagram – Logic Expression – Truth Table

- Derive the Truth Table and the Circuit Diagram for the following expression: $F(X, Y, Z) = XY + \bar{Y}Z$

Circuit diagram – Logic Expression – Truth Table

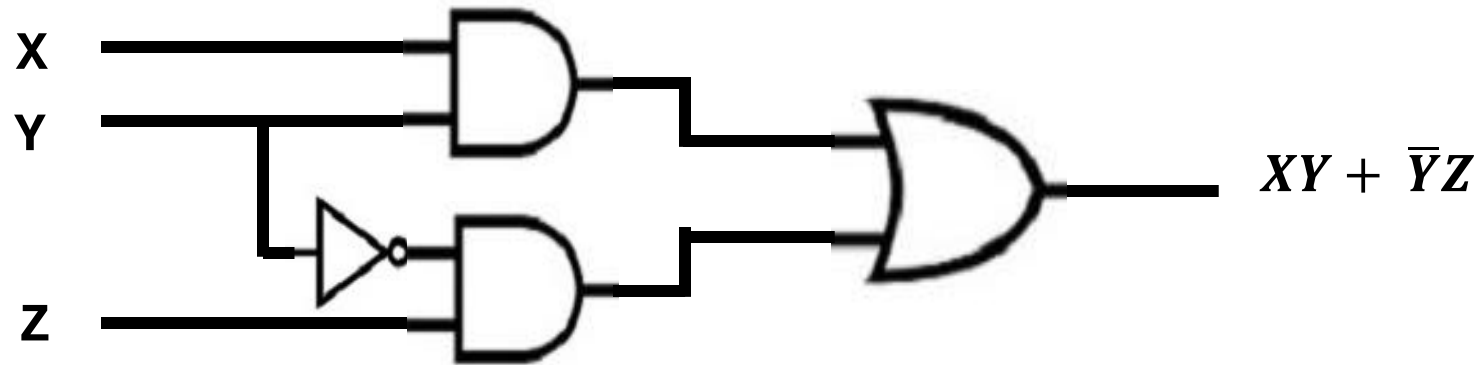
- Derive the Truth Table and the Circuit Diagram for the following expression: $F(X, Y, Z) = XY + \bar{Y}Z$

X	Y	Z	XY	$\bar{Y}$	$\bar{Y}Z$	$F = XY + \bar{Y}Z$
0	0	0	0	1	0	0
0	0	1	0	1	1	1
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	1	0	0
1	0	1	0	1	1	1
1	1	0	1	0	0	1
1	1	1	1	0	0	1

- Next, draw the circuit diagram....

Circuit diagram – Logic Expression – Truth Table

- Derive the Truth Table and the Circuit Diagram for the following expression: $F(X, Y, Z) = XY + \bar{Y}Z$



Circuit diagram – Logic Expression – Truth Table

- Using the following circuit Truth Table, derive the logical expression and draw the circuit diagram.

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Circuit diagram – Logic Expression – Truth Table

- Using the following circuit Truth Table, derive the logical expression and draw the circuit diagram.

$$F(x, y, z) = \sum m(1, 4, 5, 6, 7) = \bar{x}\bar{y}z + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z} + xyz.$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

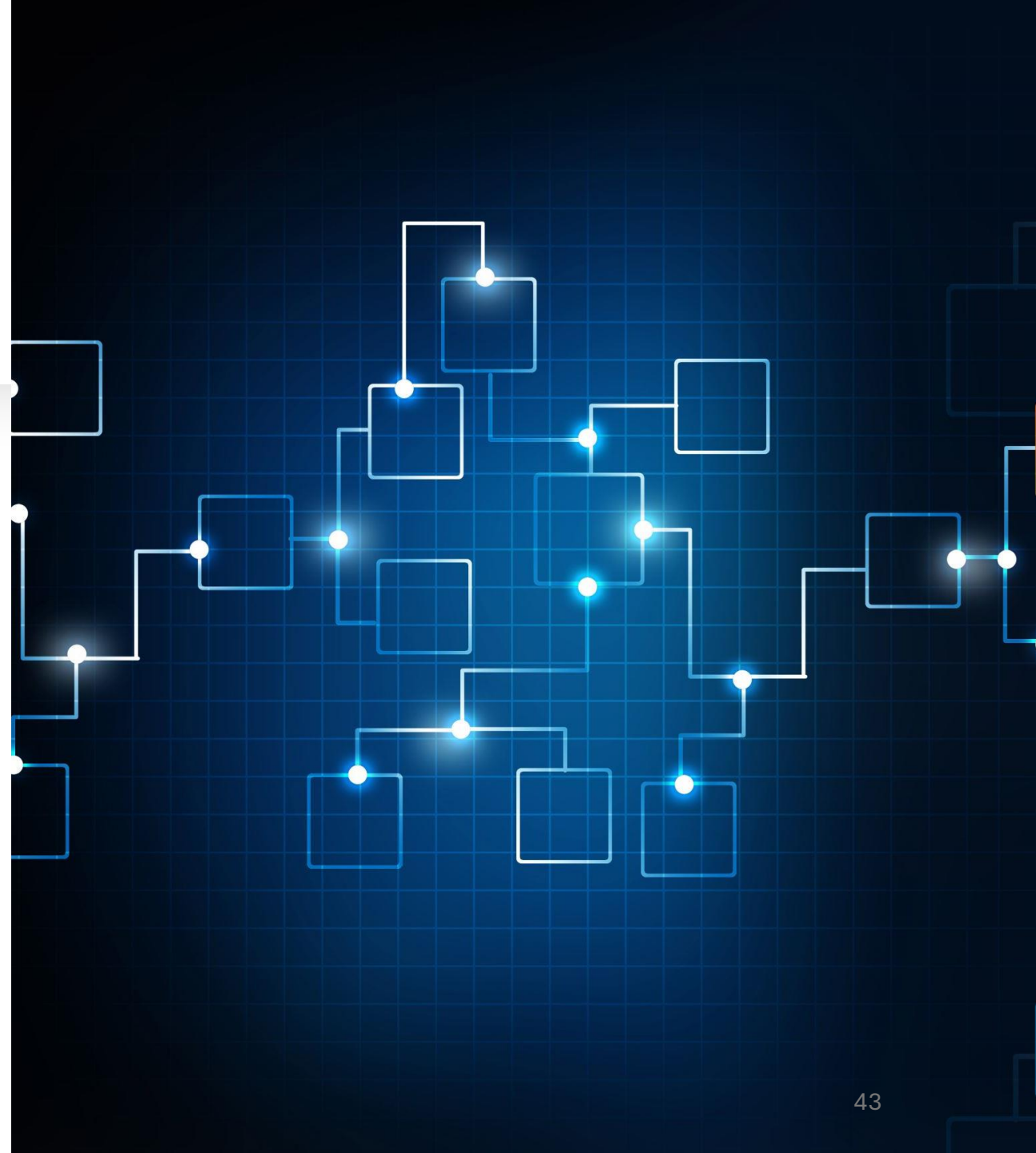
- Now try the circuit diagram....
- Simplified, this is

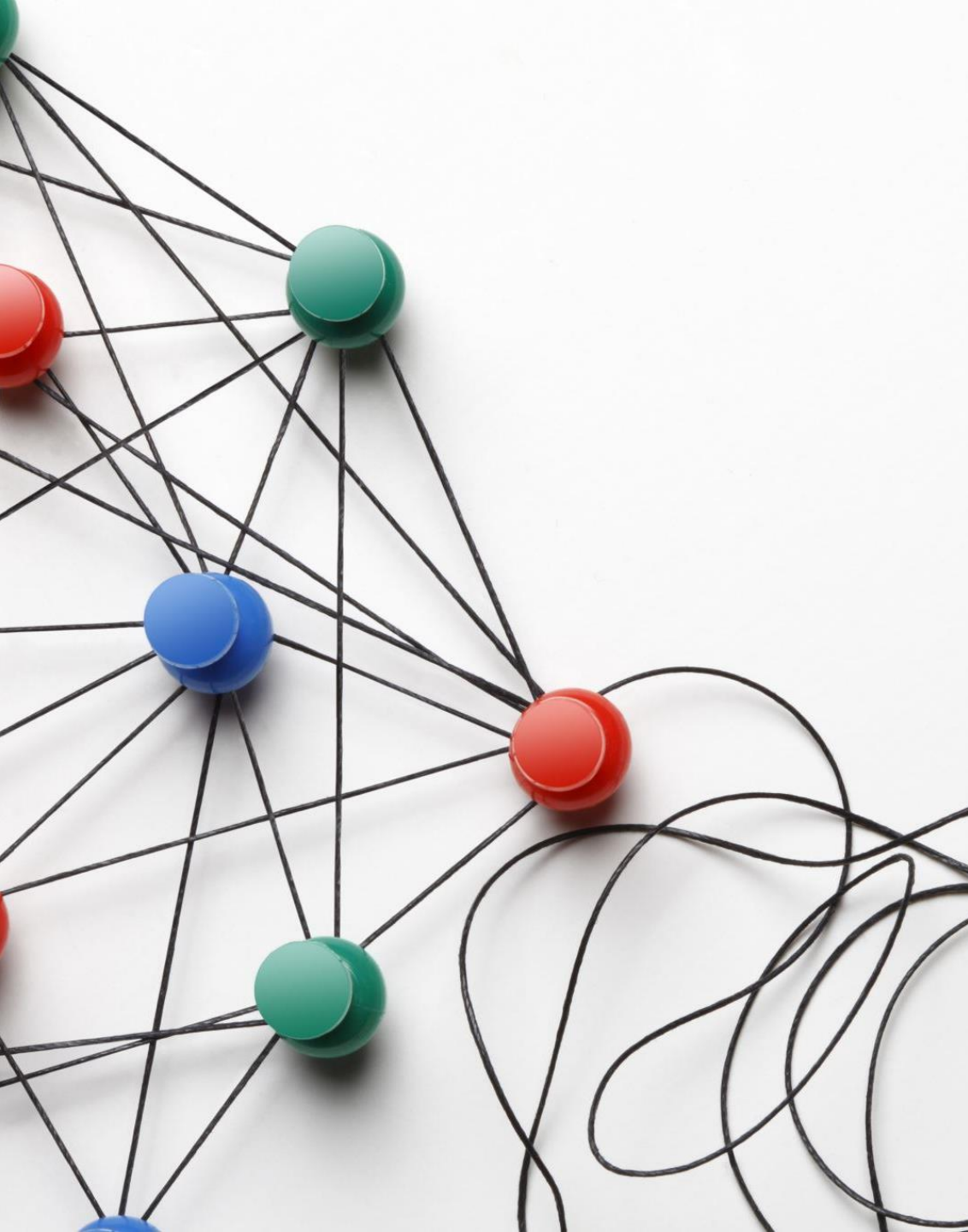
$$F = x + \bar{x}\bar{y}z$$

- We will now look at some Boolean Algebra which makes this simplification possible

Recap

- Binary signal representation (0 / 1 levels)
- Truth tables and how to construct them
- Basic logic gates: AND, OR, NOT
- Derived gates: XOR, NAND, NOR, XNOR
- Converting Truth Table $\leftrightarrow$ Expression $\leftrightarrow$ Circuit





Session Topics

- Apply basic Boolean theorems for simplification.
- Introduction to Karnaugh Maps (K-Maps)
 - 2-variable K-Maps
 - 3-variable K-Maps
 - 4-variable K-Maps
- Grouping Rules & Power-of-Two Blocks
- Wrap-Around Adjacency & Gray Code Ordering
- Handling Don't Care Conditions

Boolean Algebra

Boolean Algebra

- Identity

$$\begin{aligned}a + 0 &= a \\ a \cdot a &= a\end{aligned}$$

- Commutativity

$$\begin{aligned}a + b &= b + a \\ a \cdot b &= b \cdot a\end{aligned}$$

- Associativity

$$\begin{aligned}a + (b + c) &= (a + b) + c \\ a \cdot (b \cdot c) &= (a \cdot b) \cdot c\end{aligned}$$

Boolean Algebra Contd

- Distributivity

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$
$$a \cdot (b + c) = a \cdot b + a \cdot c$$

- Compliment

$$a + \bar{a} = 1$$
$$a \cdot \bar{a} = 0$$

- Also,

$$a + a = a$$

$$a \cdot a = a$$

$$a + 1 = 1$$

$$a \cdot 0 = 0$$

$$\bar{\bar{a}} = a$$

Boolean Algebra Contd

- Absorption:

$$a + ab = a$$
$$a(a + b) = a$$

Examples:

$$(X + Y) + (X + Y)Z = ?$$
$$A\bar{B}(A\bar{B} + \bar{B}C) = ?$$

Boolean Algebra Contd

- Absorption:

$$a + ab = a$$
$$a(a + b) = a$$

Examples:

$$(X + Y) + (X + Y)Z = X + Y$$
$$A\bar{B}(A\bar{B} + \bar{B}C) = A\bar{B}$$

DeMorgan's Law

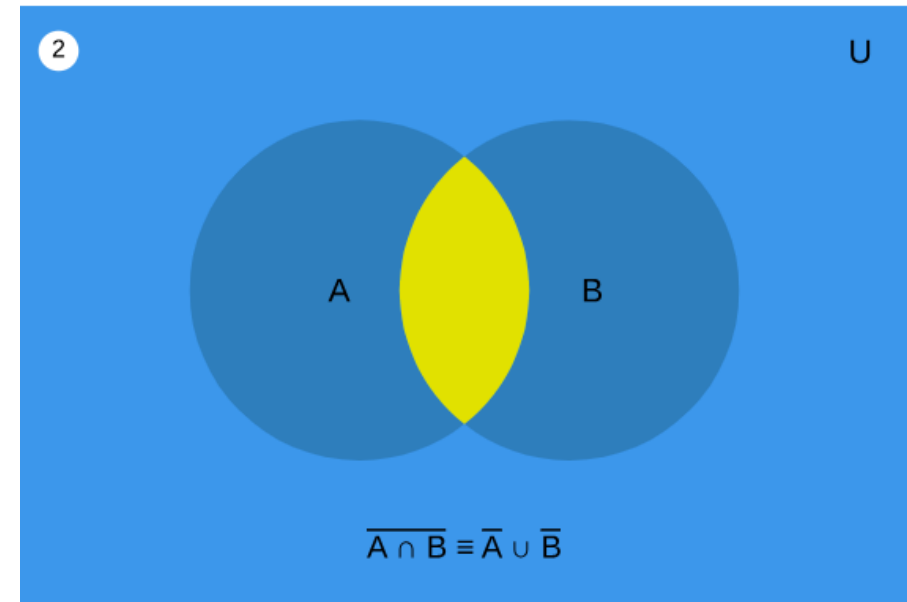
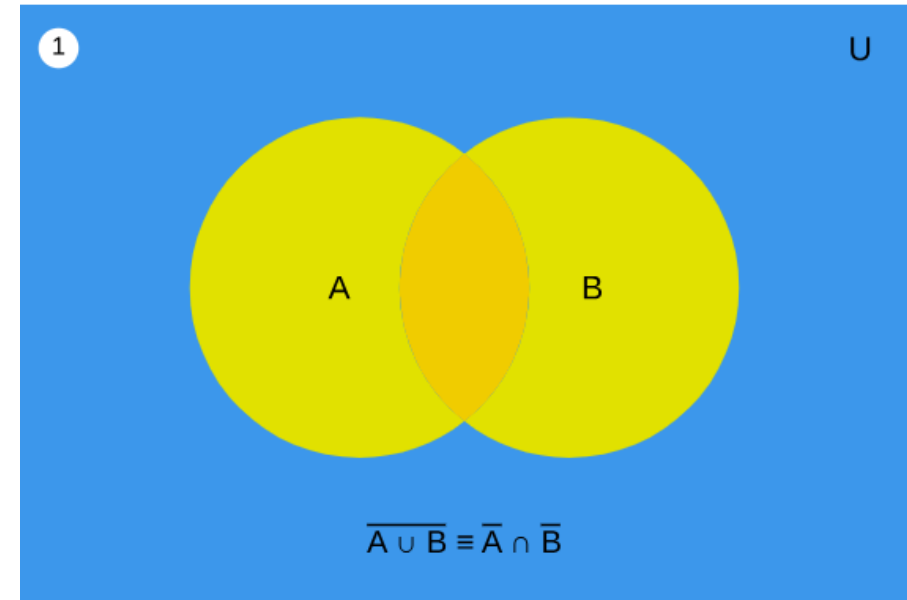
- DeMorgan Law:

$$\overline{(a \cdot b)} = \bar{a} + \bar{b}$$
$$\overline{(a + b)} = \bar{a} \cdot \bar{b}$$

- Generalized DeMorgan's Law:

$$\overline{(a + b + \cdots + z)} = \bar{a} \bar{b} \cdots \bar{z}$$

$$\overline{(a \cdot b \cdots z)} = \bar{a} + \bar{b} + \cdots + \bar{z}$$



DeMorgan's Law

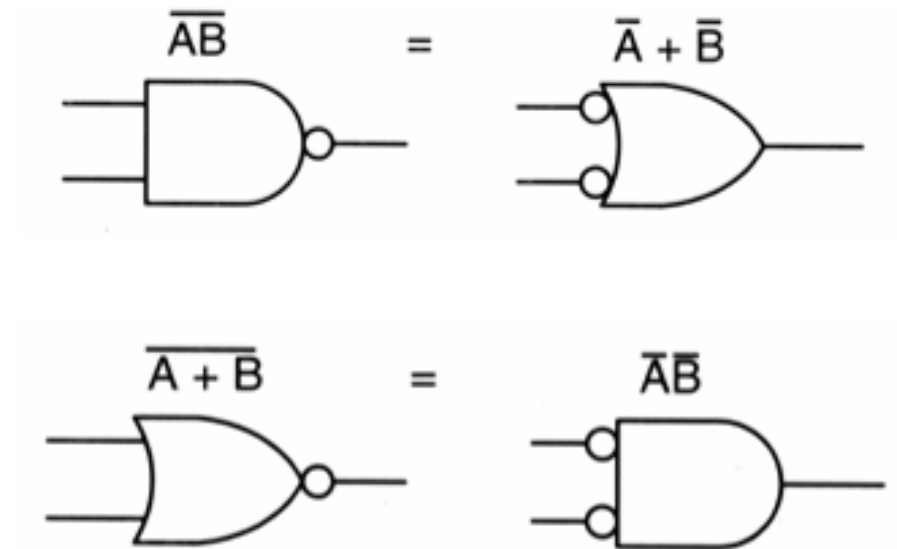
$$\overline{X + Y} = \bar{X} \cdot \bar{Y}$$

$$\overline{X \cdot Y} = \bar{X} + \bar{Y}$$

X	Y	$X \cdot Y$	$X + Y$	$\bar{X}$	$\bar{Y}$	$\overline{X + Y}$	$\bar{X} \cdot \bar{Y}$	$\overline{X \cdot Y}$	$\bar{X} + \bar{Y}$
0	0	0	0	1	1	1	1	1	1
0	1	0	1	1	0	0	0	1	1
1	0	0	1	0	1	0	0	1	1
1	1	1	1	0	0	0	0	0	0

DeMorgan's Law - Graphically

- For many people it makes sense to think about De Morgan's law graphically
- Consider the equivalence of the logic circuits involved



Simplifying Boolean Expressions

- Simplify the following Boolean Expression:

$$F = A + A \cdot B$$

Using Absorption Law,

$$A + A \cdot B = A$$

Simplifying Boolean Expressions

- Simplify the following Boolean Expression:

$$F = A \cdot B + \overline{A} \cdot B$$

Factor B,

$$F = (A + \overline{A}) \cdot B$$

Apply Compliment Law

$$F = B$$

Simplifying Boolean Expressions

- Simplify the following Boolean Expression:

$$F = A \cdot B + A \cdot \overline{B} + \overline{A} \cdot B$$

Grouping common Factor and Apply Compliment Law,

$$F = A(B + \overline{B}) + \overline{A} \cdot B$$

$$F = A + \overline{A} \cdot B$$

Simplifying Boolean Expressions

- Simplify the following Boolean Expression:

$$F = W + \overline{(\bar{X} \cdot \bar{Y}) + (\bar{X} \cdot \bar{Z})}$$

- De-Morgan's Law,

$$= W + \overline{(\bar{X} \cdot \bar{Y})} \cdot \overline{(\bar{X} \cdot \bar{Z})}$$

- De-Morgan's Law,

$$= W + (\bar{\bar{X}} + \bar{\bar{Y}}) \cdot (\bar{\bar{X}} + \bar{\bar{Z}})$$

$$= W + (X + Y) \cdot (X + Z)$$

Simplifying Boolean Expressions

$$F = W + (X + Y).(X + Z)$$

- Distributive Law,

$$= W + (X + Y)X + (X + Y)Z$$

- Distributive Law,

$$= W + X.X + Y.X + X.Z + Y.Z$$

- Absorption Law,

$$= W + X + X.Z + Y.Z$$

- Absorption Law,

$$= W + X + Y.Z$$

Simplifying Boolean Expressions

- Try simplifying this...

$$F = \overline{(X + Y)} \cdot \overline{(\bar{X} + Y)}$$

Simplifying Boolean Expressions

- Try simplifying this...

$$F = \overline{(X + Y)} \cdot \overline{(\bar{X} + Y)}$$

$$= (\bar{X} \bar{Y}) \cdot \overline{(\bar{X} + Y)}$$

$$= (\bar{X} \bar{Y}) \cdot (X \bar{Y})$$

$$= \bar{X} \bar{Y} X \bar{Y}$$

$$= (\bar{X} X) (\bar{Y} \bar{Y})$$

$$= 0 \cdot \bar{Y}$$

$$= 0$$

Simplifying Boolean Expressions

- Try simplifying this...

$$F = X + \bar{Y} + \bar{X}.Y + (X + \bar{Y}).\bar{X}.Y$$

$$F = X + \bar{Y} + \bar{X}Y + X\bar{X}Y + \bar{Y}\bar{X}Y$$

$$X\bar{X} = 0, \quad Y\bar{Y} = 0$$

$$F = X + \bar{Y} + \bar{X}Y + 0 + 0$$

$$F = X + \bar{Y} + \bar{X}Y$$

Simplifying Boolean Expressions

$$F = X + \overline{Y} + \overline{X}Y$$

Factor with the distributive identity $A + BC = (A + B)(A + C)$

$$= X + (\overline{Y} + \overline{X})(\overline{Y} + Y)$$

$$= X + \overline{Y} + \overline{X}$$

$$= X + \overline{X} + \overline{Y}$$

$$= 1 + \overline{Y}$$

$$= 1$$

Karnaugh Maps: Fundamentals & Grouping

Karnaugh maps

- The Karnaugh map (K-map) is a better way to simplify Boolean algebra expressions.
- K-maps reduce the need for a lot of calculation, using simple pattern recognition (visual method) ability in us.
- It groups adjacent 1s to form simplified product terms.
- Only groups of size 1, 2, 4, 8... (powers of 2) are allowed.
- Larger groups → More simplification.

Two-Variable K-maps

Truth Table

A	B	
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

Karnaugh map

		A	
		0	1
B	0	m_0	m_2
	1	m_1	m_3

- So, you can see that they are the same, just two different representations.

Two-Variable K-maps

- Draw the map for the following truth table,

	A	B	
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

		A	
		0	1
B	0	☐	☐
	1	☐	☐

Two-Variable K-maps

	A	B	
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

	A	
B	0	1
0	m_0	m_2
1	m_1	m_3

Two-Variable K-maps

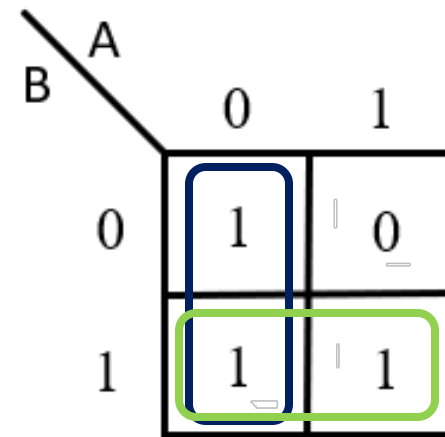
	A	B	
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

		A	
		0	1
B	0	1	0
	1	1	1

Two-Variable K-maps

- Next step is to “Group” the “1”s together... (groups of 1,2,4,8..)

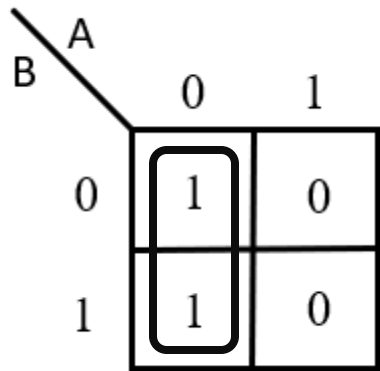
	A	B	
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1



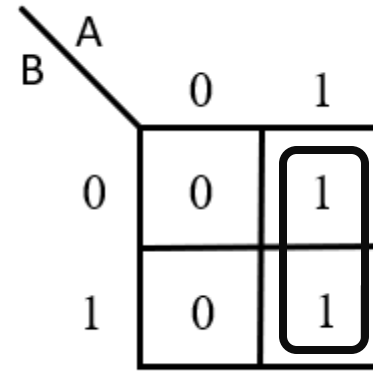
$$\bar{A} + B$$

Two-Variable K-maps

- Finding the right expression.

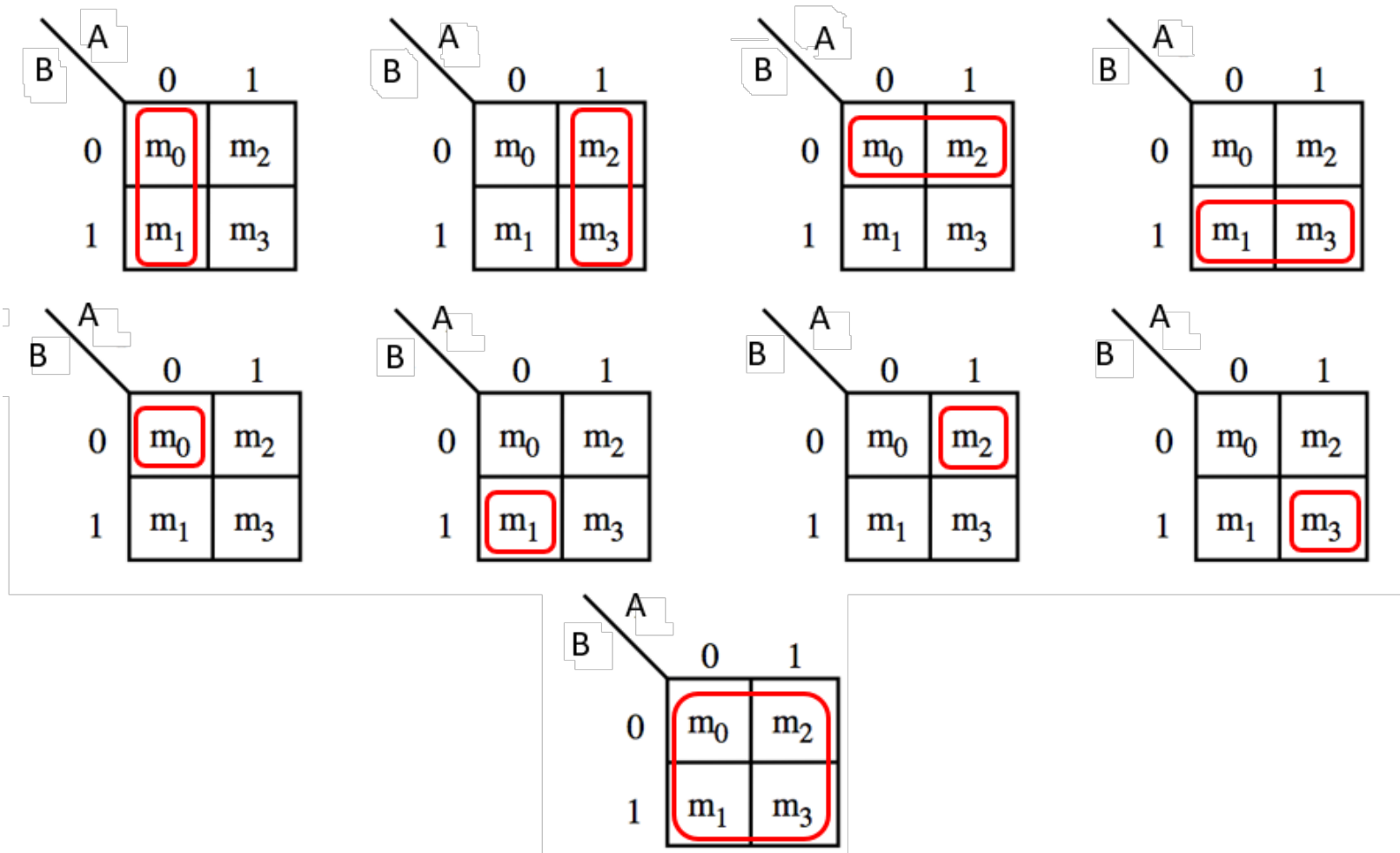


$\bar{A}$ is constant



A is constant

Valid Groupings



Invalid Grouping

		A	
		0	1
B	0	m ₀	m ₂
	1	m ₁	m ₃

		A	
		0	1
B	0	m ₀	m ₂
	1	m ₁	m ₃

		A	
		0	1
B	0	1	0
	1	0	1

$$m_0 + m_3$$

$$\bar{A}\bar{B} + AB$$

Three-Variable K-Maps

- We will use inputs as x_1, x_2 and x_3

Did you notice something strange about the Placement of Variables?

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Three-Variable K-Maps

- We will use inputs as x_1, x_2 and x_3

Did you notice something strange about the Placement of Variables?

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

$x_1 x_2$		x_3			
		00	01	11	10
0	1	m_0	m_2	m_6	m_4
		m_1	m_3	m_7	m_5

(b) Karnaugh map

Three-Variable K-Maps

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Three-Variable K-Maps

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

$x_1 x_2$		x_3			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Gray Coding, Adjacency & Don't Care Conditions

Gray Coding

- Gray coding (or Gray code) in Karnaugh maps (K-maps) refers to the specific ordering of variables along the rows and columns such that only one-bit changes between adjacent cells.

Why is Gray Coding Used in K-Maps?

- In Boolean simplification, we group adjacent 1s (or 0s) in powers of 2.
- To ensure that adjacent cells represent minterms that differ by only one variable, K-maps use Gray code ordering, not normal binary counting.

Gray Coding

- Example: 2-bit Gray Code

Decimal	Binary	Gray Code
0	00	00
1	01	01
2	10	11
3	11	10

Gray Coding

- Example: 3-bit Gray Code

Decimal	Binary	Gray Code
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Gray Coding & K-Maps

	$x_1x_2x_3$
m_0	0 0 0
m_1	0 0 1
m_2	0 1 0
m_3	0 1 1
m_4	1 0 0
m_5	1 0 1
m_6	1 1 0
m_7	1 1 1

		x_1x_2			
		00	01	11	10
x_3	0	000	010	110	100
	1	001	011	111	101

Gray Coding & K-Maps

	$x_1x_2x_3$
m_0	0 0 0
m_1	0 0 1
m_2	0 1 0
m_3	0 1 1
m_4	1 0 0
m_5	1 0 1
m_6	1 1 0
m_7	1 1 1

$$F = \overline{x_1} \cdot \overline{x_2}$$

		x_1x_2			
		00	01	11	10
x_3	0	000	010	110	100
	1	001	011	111	101

Notice that there is only one bit (Last bit) flip between neighbors.

Gray Coding & K-Maps

	$x_1 x_2 x_3$
m_0	0 0 0
m_1	0 0 1
m_2	0 1 0
m_3	0 1 1
m_4	1 0 0
m_5	1 0 1
m_6	1 1 0
m_7	1 1 1

$$F = x_1 \cdot \overline{x_2}$$

		$x_1 x_2$			
		00	01	11	10
x_3	0	000	010	110	100
	1	001	011	111	101

Notice that there is only one bit (Last bit) flip between neighbors.

Gray Coding & K-Maps

	$x_1x_2x_3$
m_0	0 0 0
m_1	0 0 1
m_2	0 1 0
m_3	0 1 1
m_4	1 0 0
m_5	1 0 1
m_6	1 1 0
m_7	1 1 1

x_1x_2		00	01	11	10
x_3	0	000	010	110	100
	1	001	011	111	101

$$F = x_2 \overline{x_3}$$

Notice that there is only one bit (First bit) flip between neighbors.

Gray Coding & K-Maps

	$x_1 x_2 x_3$
m_0	0 0 0
m_1	0 0 1
m_2	0 1 0
m_3	0 1 1
m_4	1 0 0
m_5	1 0 1
m_6	1 1 0
m_7	1 1 1

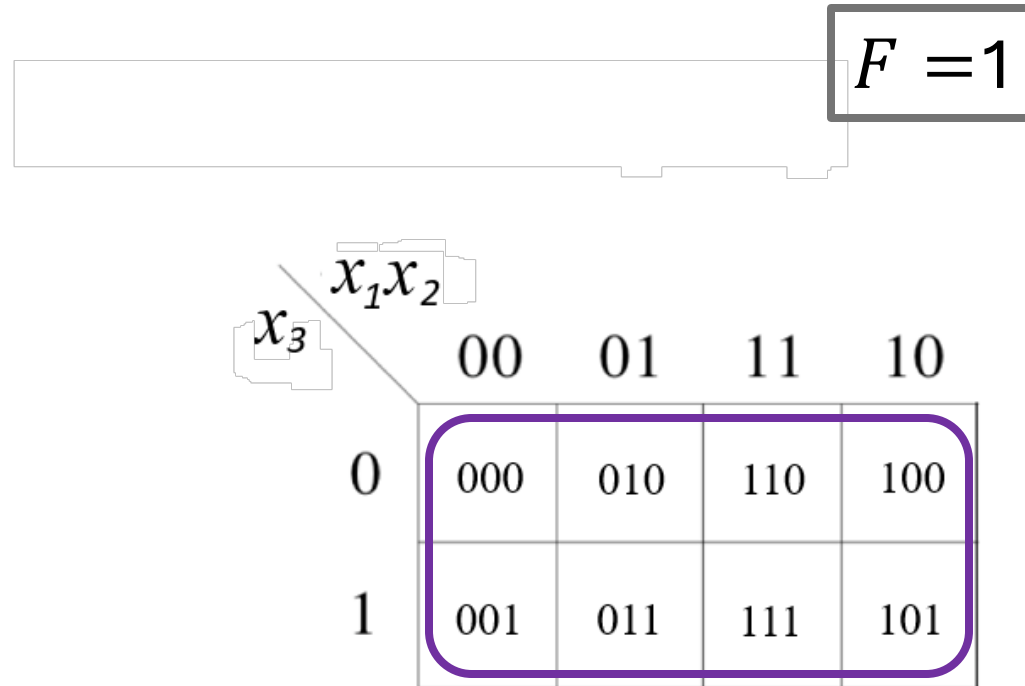
$x_3 \backslash x_1 x_2$	00	01	11	10
0	000	010	110	100
1	001	011	111	101

$$F = x_2$$

Notice that there is a 2-bit change while,
Middle bit is the same
More components considered means a simpler expression

Gray Coding & K-Maps

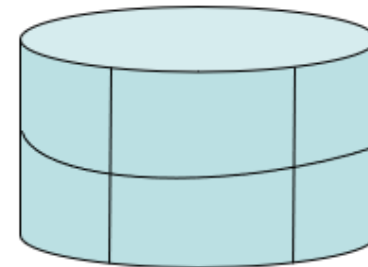
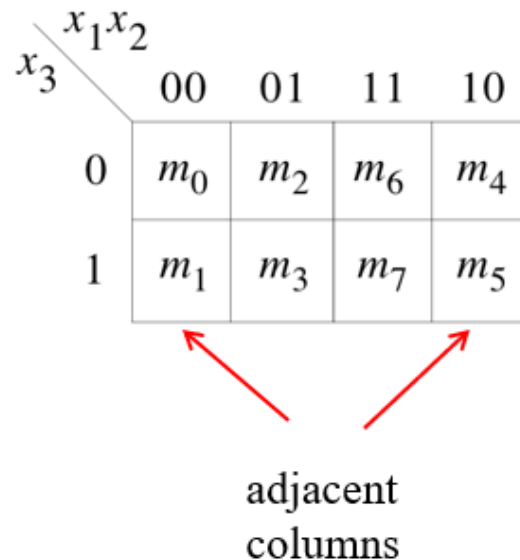
	$x_1 x_2 x_3$
m_0	0 0 0
m_1	0 0 1
m_2	0 1 0
m_3	0 1 1
m_4	1 0 0
m_5	1 0 1
m_6	1 1 0
m_7	1 1 1



What if all were “1”...?

Adjacency Rules

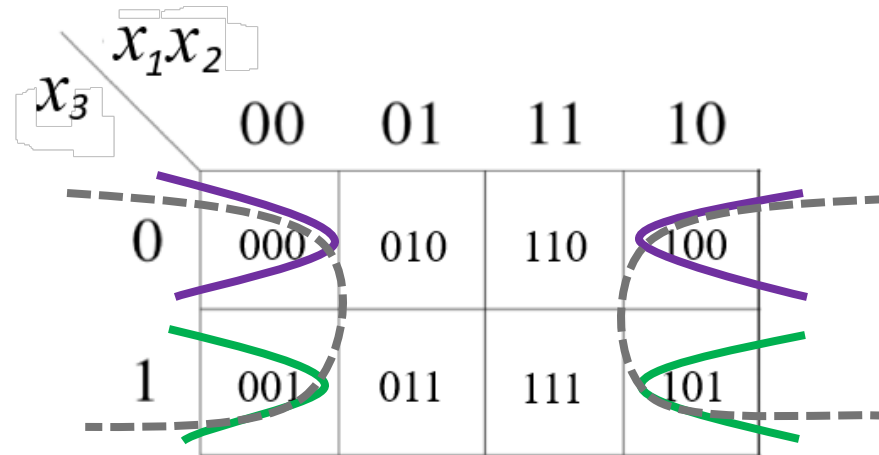
- Two or more K-map cells are considered **adjacent** if **they differ by only ONE variable** (i.e., their binary representations differ by only one bit — following Gray code ordering).
- This adjacency applies:
- **Horizontally** (left–right neighbors)
- **Vertically** (top–bottom neighbors)
- **Edge wrapping** — *cells on opposite edges are also adjacent!*



As if the K-map were drawn on a cylinder

Adjacency Rules

	$x_1x_2x_3$
m_0	0 0 0
m_1	0 0 1
m_2	0 1 0
m_3	0 1 1
m_4	1 0 0
m_5	1 0 1
m_6	1 1 0
m_7	1 1 1



Same principle applies for both ROWS and COLUMNS

Valid Grouping Examples

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

Valid Grouping Examples

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

Valid Grouping Examples

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

Valid Grouping Examples

x_3	x_1x_2			
	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

x_3	x_1x_2			
	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

x_3	x_1x_2			
	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

INVALID

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

		x_1x_2			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

Four-Variable K-Map

x1	x2	x3	x4	
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15

		$x_1 x_2$			
		$x_3 x_4$			
		00	01	11	10
00		m_0	m_4	m_{12}	m_8
01		m_1	m_5	m_{13}	m_9
11		m_3	m_7	m_{15}	m_{11}
10		m_2	m_6	m_{14}	m_{10}

Valid Grouping Examples

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	

Groupings: x_3 (rows 11, 10), x_4 (rows 01, 11), x_2 (columns 11, 10)

$x_3x_4 \backslash x_1x_2$		$\sim x_1$			
		00	01	11	10
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	

Groupings: x_3 (rows 11, 10), x_2 (columns 11, 10)

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	

Groupings: x_3 (rows 11, 10), x_4 (rows 01, 11), x_2 (columns 11, 10)

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
00	m_0	m_4	m_{12}	m_8	
01	m_1	m_5	m_{13}	m_9	
11	m_3	m_7	m_{15}	m_{11}	
10	m_2	m_6	m_{14}	m_{10}	

Groupings: x_3 (rows 11, 10), x_2 (columns 11, 10)

Valid Grouping Examples

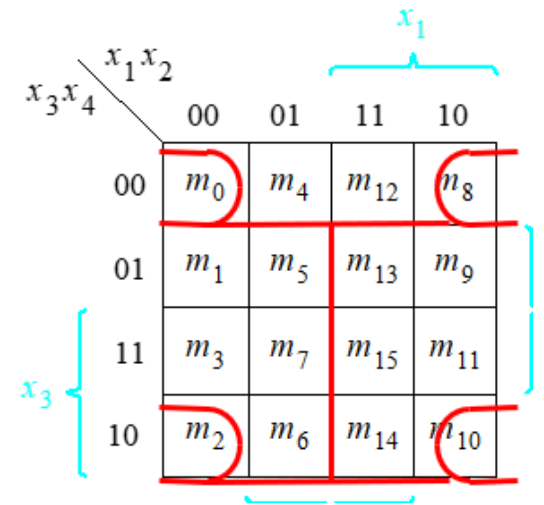
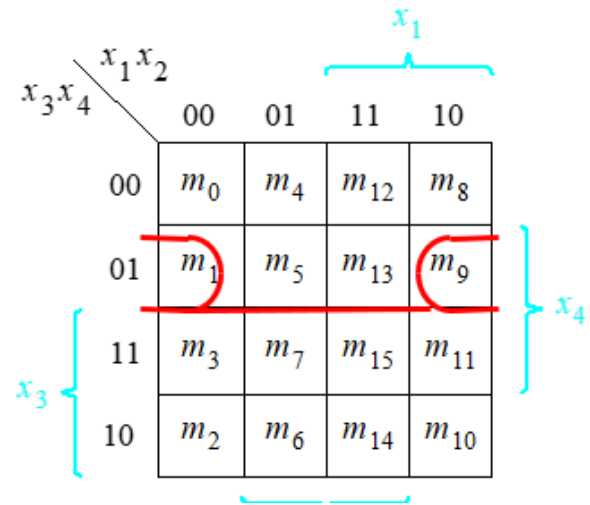
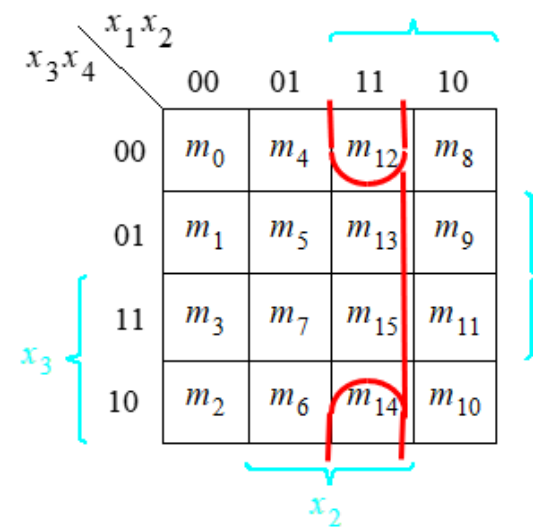
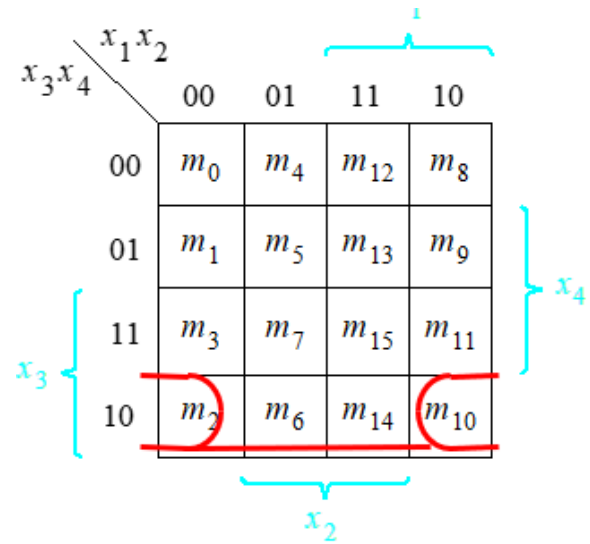
		$x_1 x_2$			
		00	01	11	10
$x_3 x_4$	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

		$x_1 x_2$			
		00	01	11	10
$x_3 x_4$	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

		$x_1 x_2$			
		00	01	11	10
$x_3 x_4$	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

		$x_1 x_2$			
		00	01	11	10
$x_3 x_4$	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

Valid Grouping Examples



INVALID

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

x_2

$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

x_2

All sides must be powers of 2.

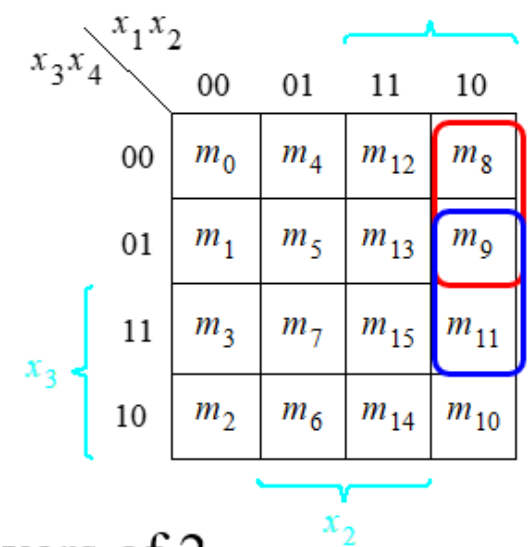
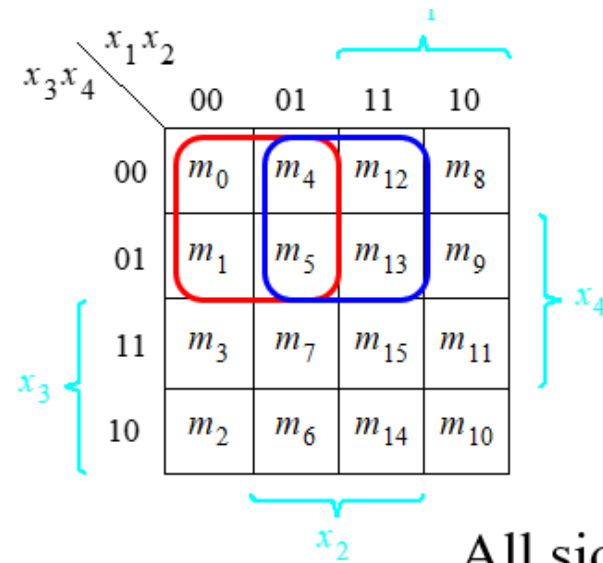
$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

x_2

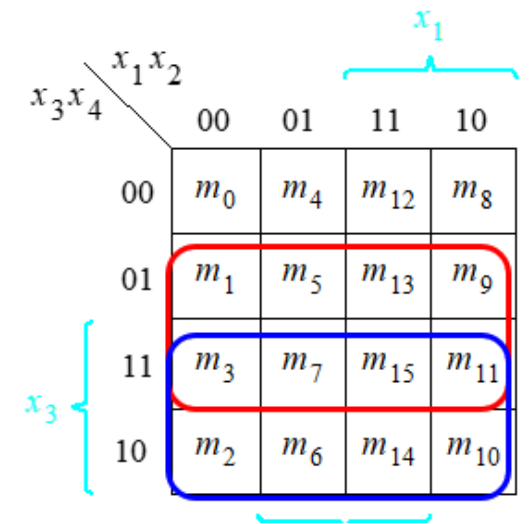
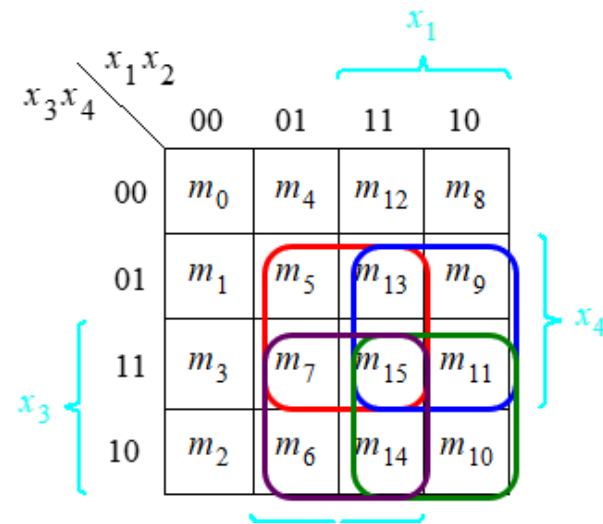
$x_3x_4 \backslash x_1x_2$		x_1			
		00	01	11	10
x_3	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

x_2

Valid Grouping Examples



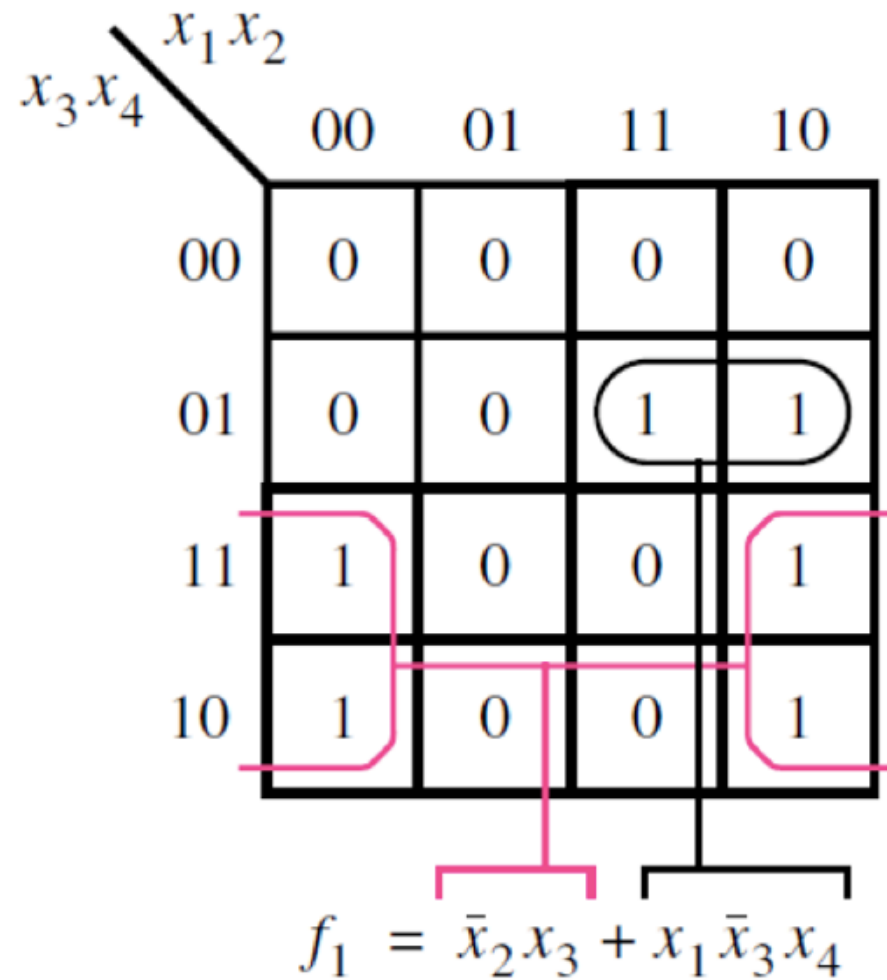
All sides must be powers of 2.



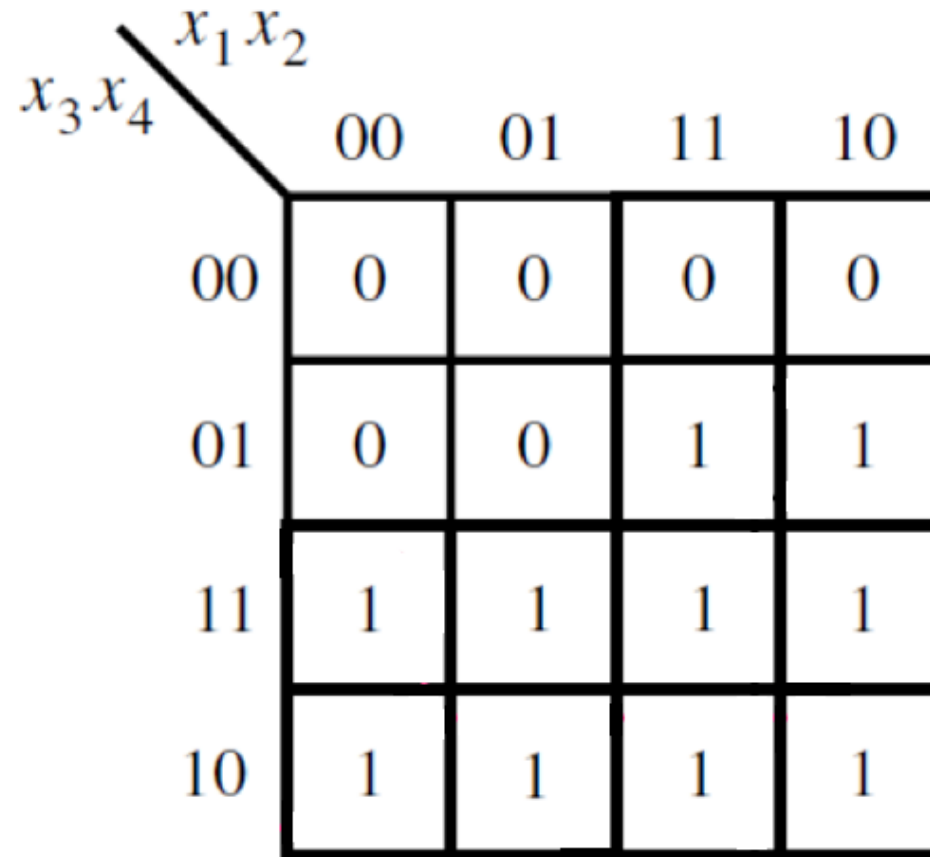
Try simplifying this K-Map

$x_1 x_2$		$x_3 x_4$			
		00	01	11	10
00	0	0	0	0	0
01	0	0	1	1	
11	1	0	0	1	
10	1	0	0	1	

Solution



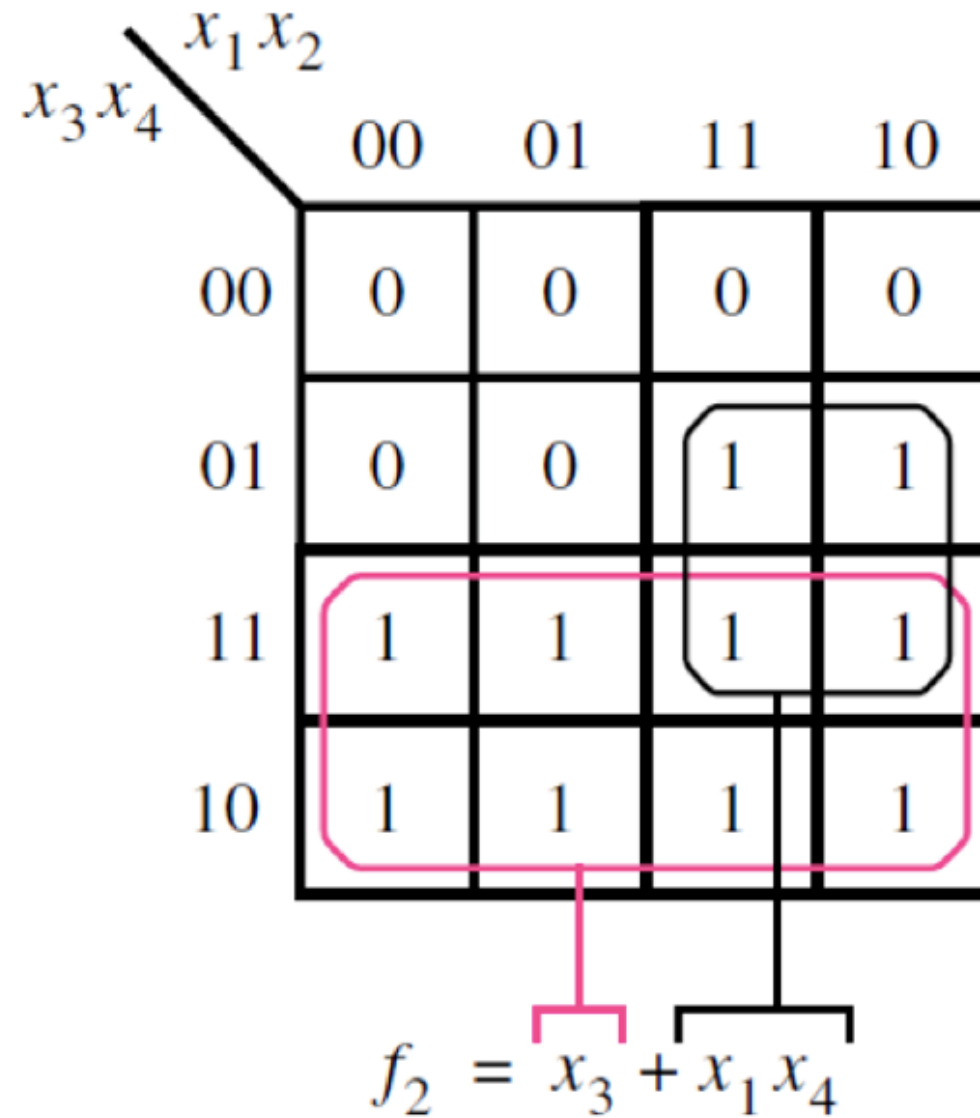
Try simplifying this K-Map



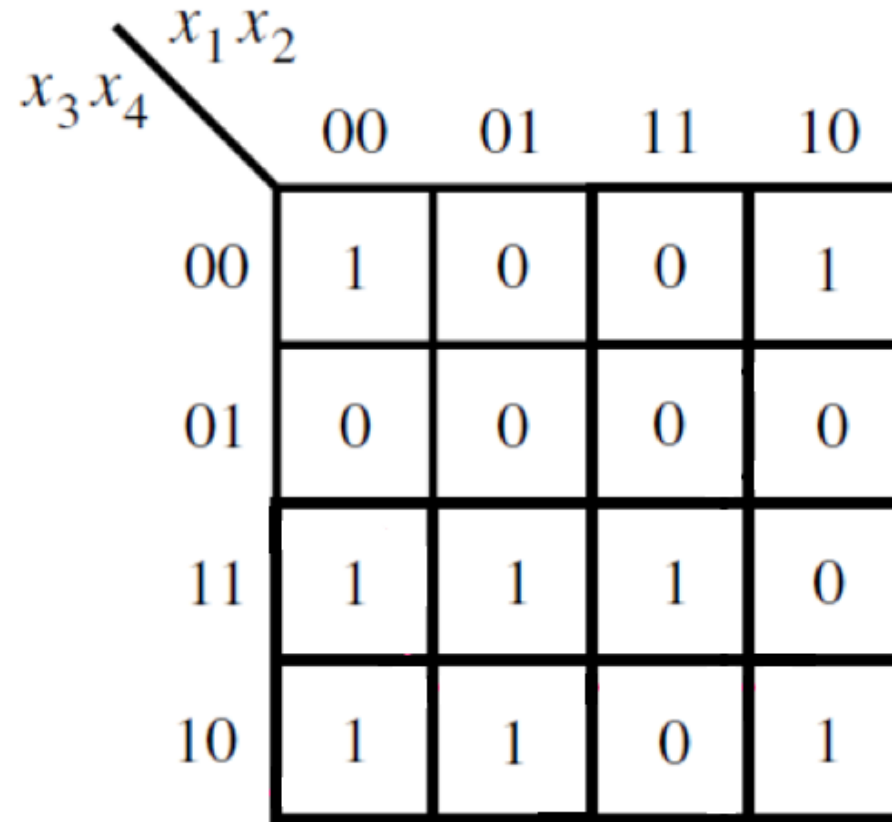
A 4x4 Karnaugh map for a 4-variable function. The horizontal axis is labeled $x_1 x_2$ with values 00, 01, 11, 10. The vertical axis is labeled $x_3 x_4$ with values 00, 01, 11, 10. The map contains the following values:

$x_3 x_4 \backslash x_1 x_2$	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	1	1	1	1
10	1	1	1	1

Solution



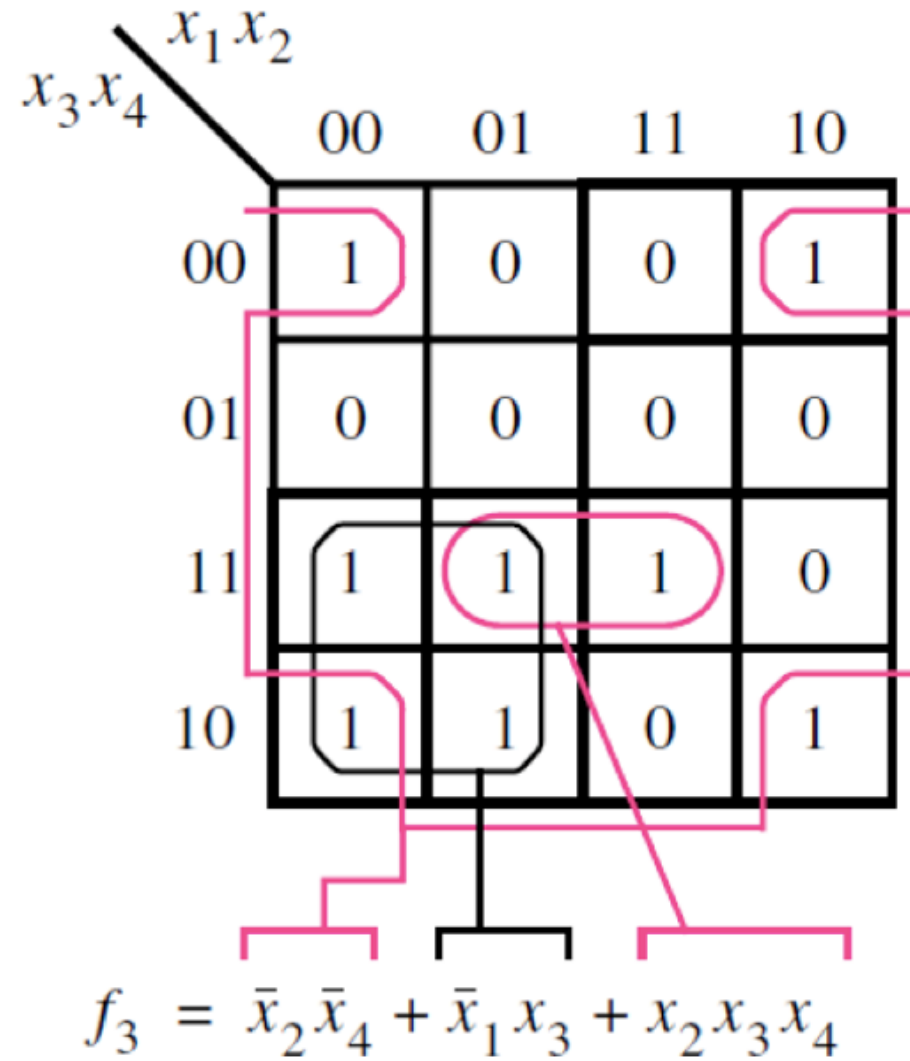
Try simplifying this K-Map



A 4x4 Karnaugh map for a 4-variable function. The horizontal axis is labeled $x_1 x_2$ with values 00, 01, 11, 10. The vertical axis is labeled $x_3 x_4$ with values 00, 01, 11, 10. The cells contain the following values:

$x_3 x_4 \backslash x_1 x_2$	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	1	1	1	0
10	1	1	0	1

Solution



DON'T CARE?

- In a Karnaugh map, a Don't Care state (usually written as X) represents an input combination where the output can be either 0 or 1 — it doesn't matter for the real-world operation of the circuit.
- These cases will never occur or are irrelevant, so we are free to treat them as 0 or 1 to simplify the expression.
- Don't care condition can allow us to create a simpler logic expression

		AB			
		00	01	11	10
CD	00	0	1	1	1
	01	X	1	1	1
	11	1	1	1	1
	10	0	1	1	1

$$f = A + B + C.D$$

		AB			
		00	01	11	10
CD	00	0	1	1	1
	01	X	1	1	1
	11	1	1	1	1
	10	0	1	1	1

$$f = A + B + D$$

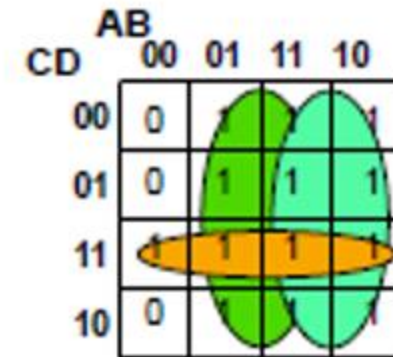
Minimising Complex Logical Expressions

- Seemingly complex logical expressions often simplify to expressions that you could implement using very few integrated circuits.

$$f = A + (\overline{B \cdot C}) + (\overline{B \cdot D})$$

Construct Truth Table:

A	B	C	D	$\overline{B \cdot C}$	$\overline{B \cdot D}$	$\overline{(\overline{B \cdot C}) + (\overline{B \cdot D})}$	f
0	0	0	0	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	0	1	1	0	0	1	1
0	1	0	0	0	0	1	1
0	1	0	1	0	0	1	1
0	1	1	0	0	0	1	1
0	1	1	1	0	0	1	1
1	0	0	0	1	1	0	1
1	0	0	1	1	0	0	1
1	0	1	0	0	1	0	1
1	0	1	1	0	0	1	1
1	1	0	0	0	0	1	1
1	1	0	1	0	0	1	1
1	1	1	0	0	0	1	1
1	1	1	1	0	0	1	1



$$f = B + A + C.D$$

Design of combinational digital circuits

- From the problem statement derive the truth table..
- (*Truth table may/may not be provided in the exam)
- Derive the simplified logic expression using K-maps (or simplify using Boolean Algebra)
- Draw the logic circuit from this solution (simplified)

Design of combinational digital circuits

- Example: Design a 3-input (A,B,C) digital circuit that would give at its output (X) a logic 1 only if the binary number formed at the input has more ones than zeros.

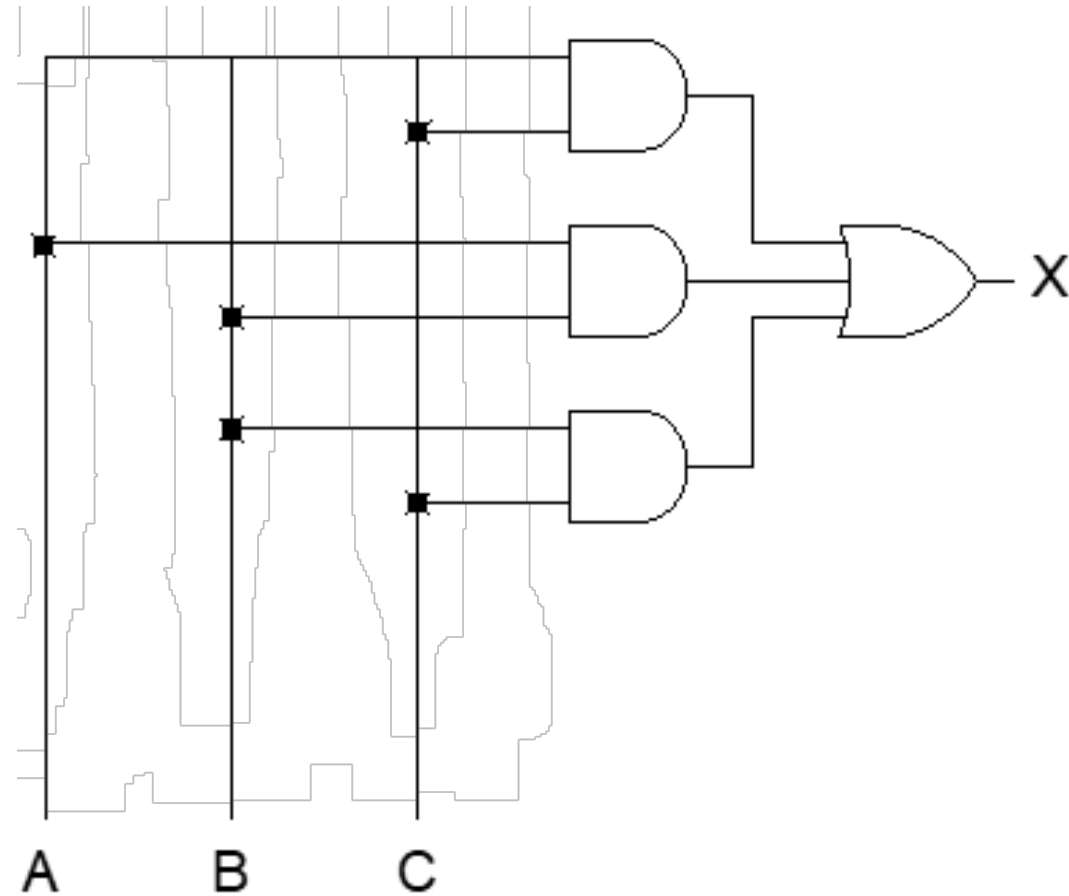
Inputs			Output
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$X = AC + AB + BC$$

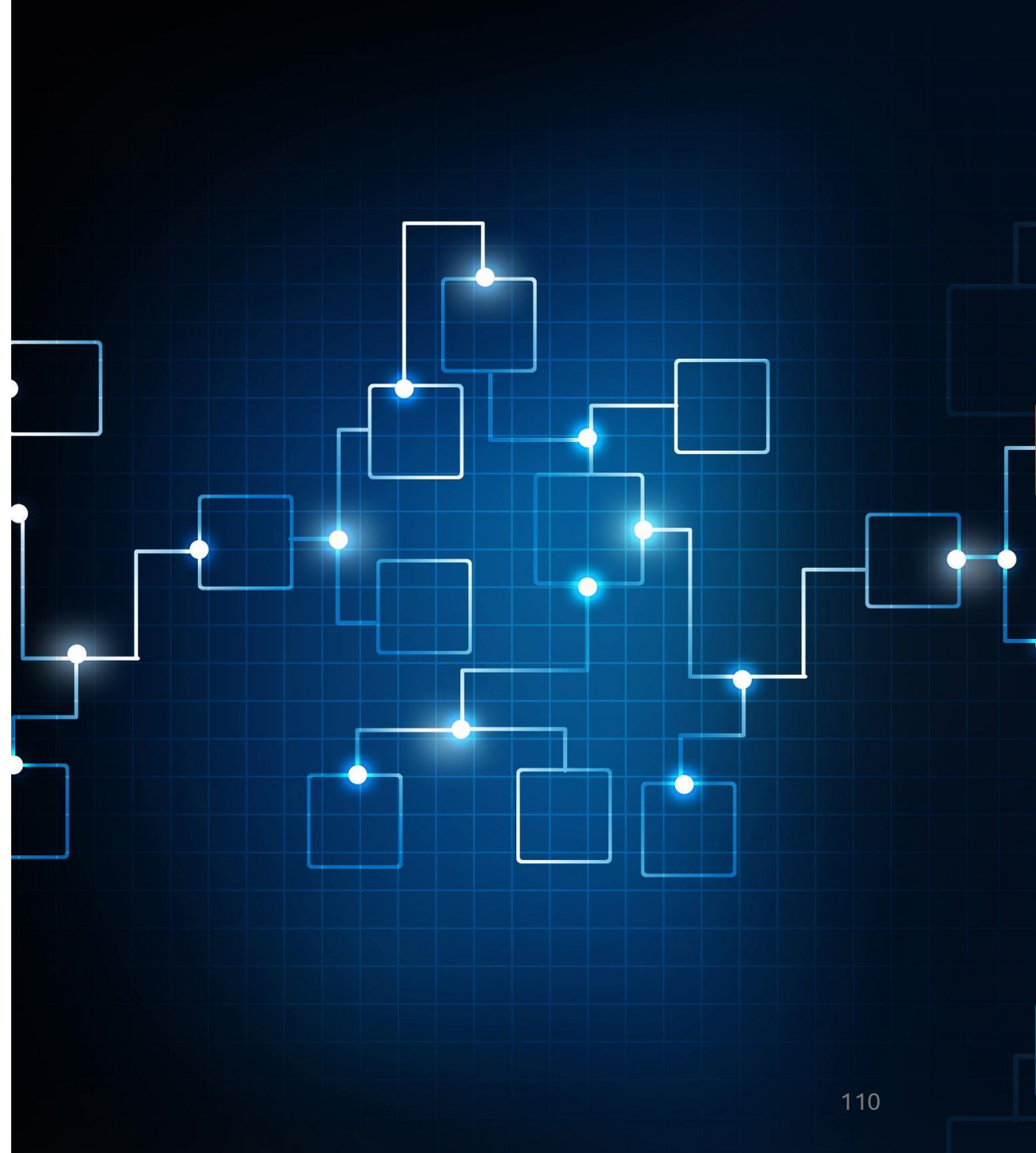
Design of combinational digital circuits

$$X = AC + AB + BC$$



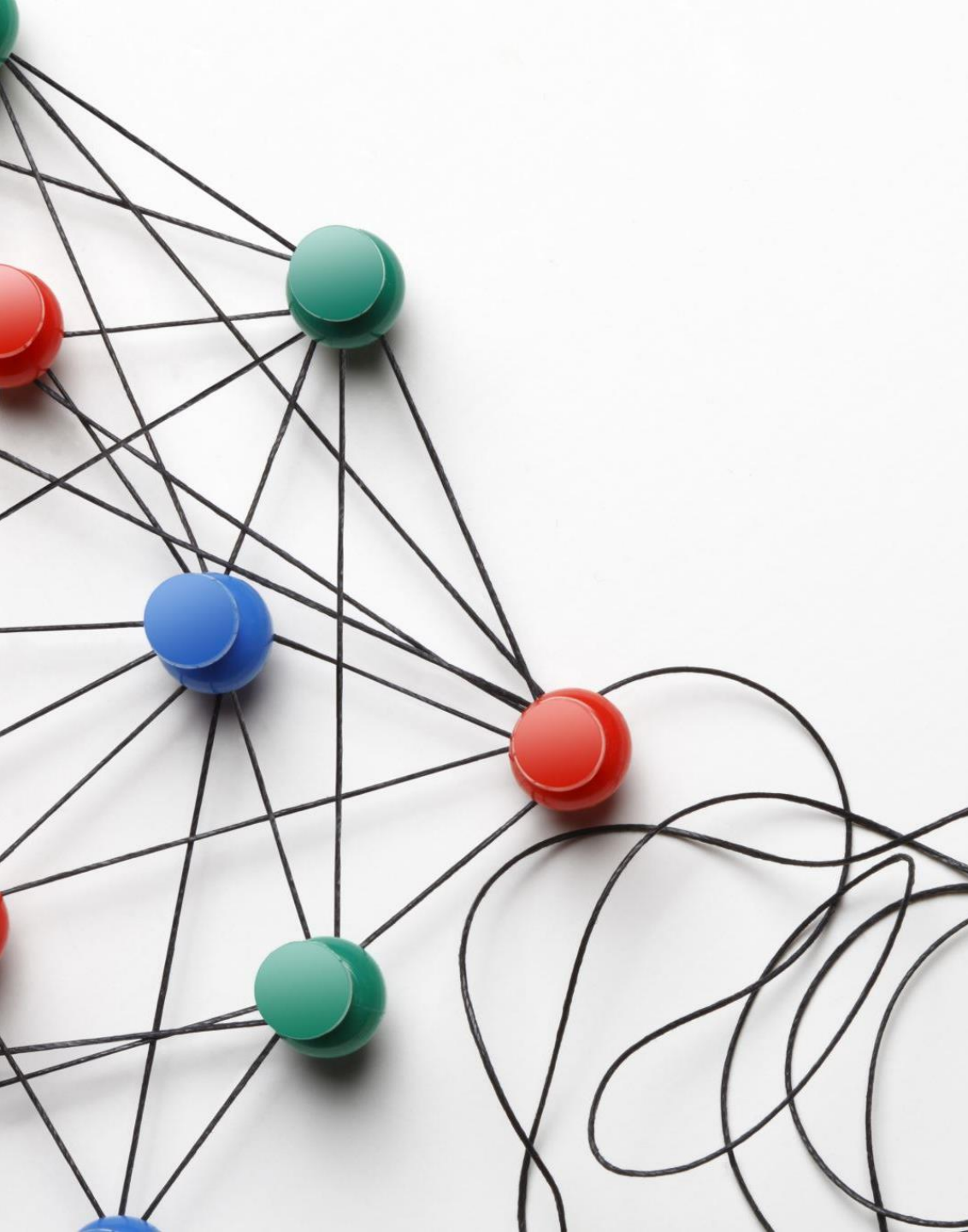
Recap

- Boolean simplification using algebraic laws
- Multi-step reduction using De Morgan + Absorption
- Introduction to Karnaugh Maps (K-Maps)
- 2-variable and 3-variable K-Map minimization
- Grouping rules and Gray code ordering
- 4-variable K-Map simplification
- Handling Don't Care (X) conditions



Topic 2 Outline

- Basic logic functions, expressions & truth tables
 - Logic gates, circuits, & finding the truth table
 - Simplifying logic expressions using Boolean algebra and Karnaugh maps
- Common combinatorial, i.e., no memory, logic circuits
 - Sequential, i.e., with memory, logic circuits



Session Topics

- Combinational Vs Sequential circuits
- Half Adder → Full Adder (2 Half Adders + OR)
- Subtraction via Two's Complement
- Adder/Subtractor circuit using XOR + Carry-in
- Transition from Combinational → Sequential Logic
- Introduction to SR / D / JK / T Flip-Flops
- Clock-controlled state-holding circuits

More Combinational Logic Circuits

- First, what are “Combinational Logic Circuits”?
- “Combinational Logic Circuits are digital circuits where the output depends only on the current input values”...
- So, all circuits we have studied so far falls into this category.
- Lets consider the EX-OR Function for an example

More Combinational Logic Circuits

EX-OR function

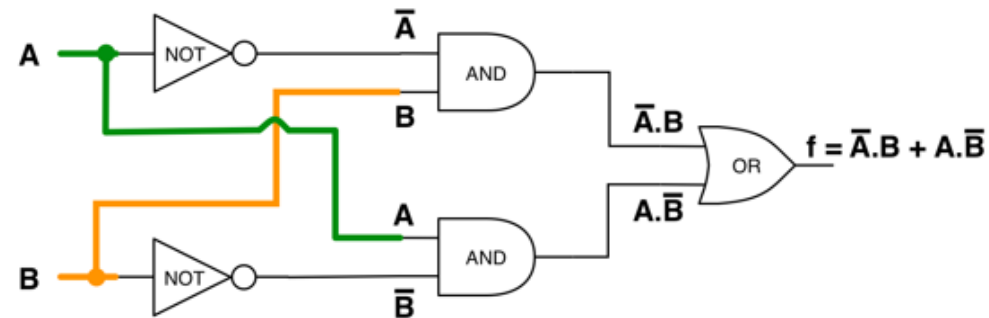
- $F = A \oplus B$

From the truth table we can write:

- $F = \bar{A}B + A\bar{B}$

- The logic circuit to the right implements the EX-OR function

A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0



- Next, lets explore some circuits developed for certain purposes.

Arithmetic Circuits: Adders, Subtraction & Active Logic

Adders

- An adder is a digital circuit that performs the addition operation on numbers (binary)
- Adders are found inside the ALU (Arithmetic Logic Unit)
- Used to calculate addresses, table indices, and similar operations.
- First let's recap on how Binary addition works for a given 2 bits.

Binary Addition

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- Observing the truth table for Sum and Carry,

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A.B$$

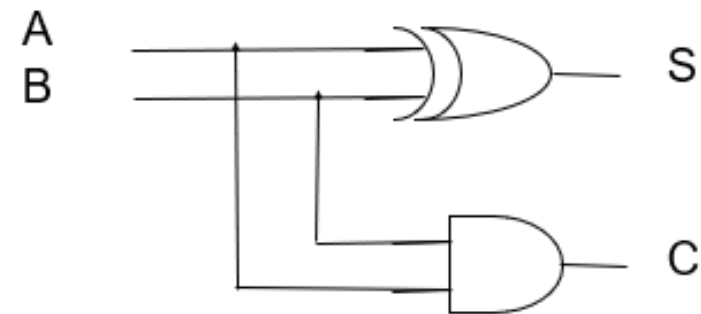
Types of Adders

- Half Adder
 - Full Adder
- Ripple Adder
 - Look Ahead Carry Unit
 - Carry-save Adder

1-bit Half Adder

- The 1-bit half-adder performs the addition of two bits
- Can be extended to form a 1-bit full-adder

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



1-bit Full Adder

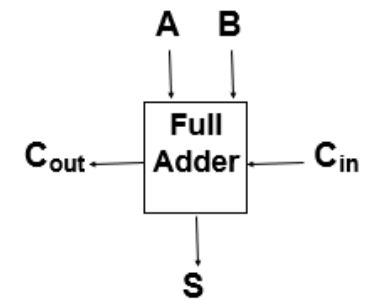
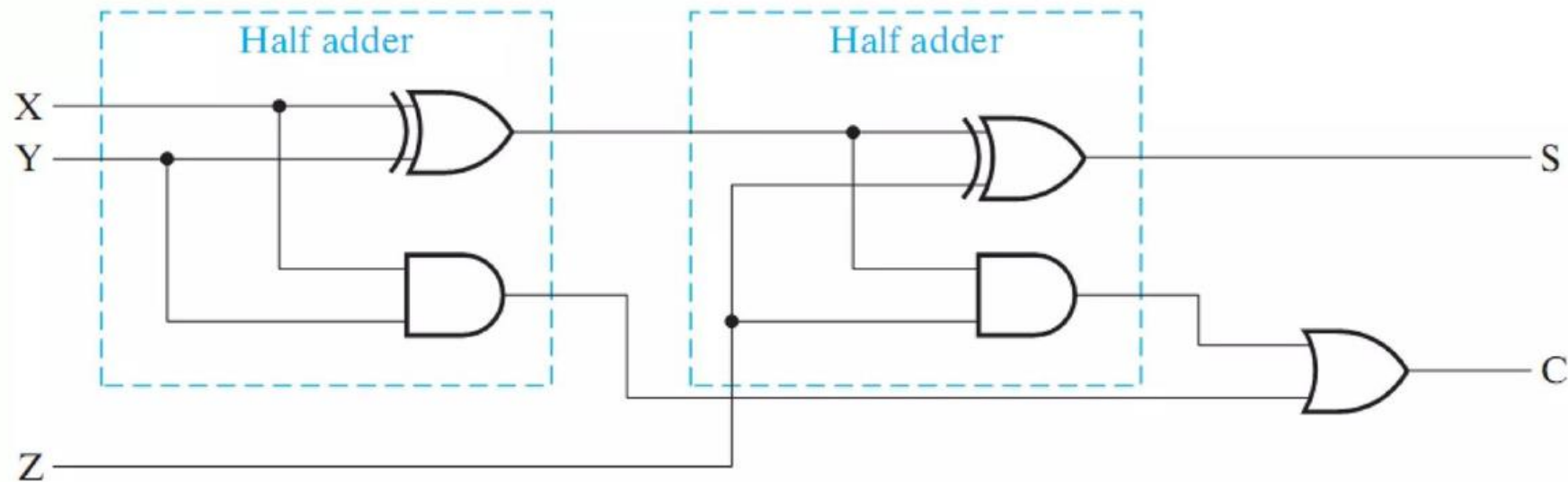
- Adds 3 inputs together.
- 3rd input can be the carry over from the previous half adder

X	Y	Z	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

1-bit Full Adder

- Basically, two half adders in series
- Try out the combinations in the table and check the answers.

X	Y	Z	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

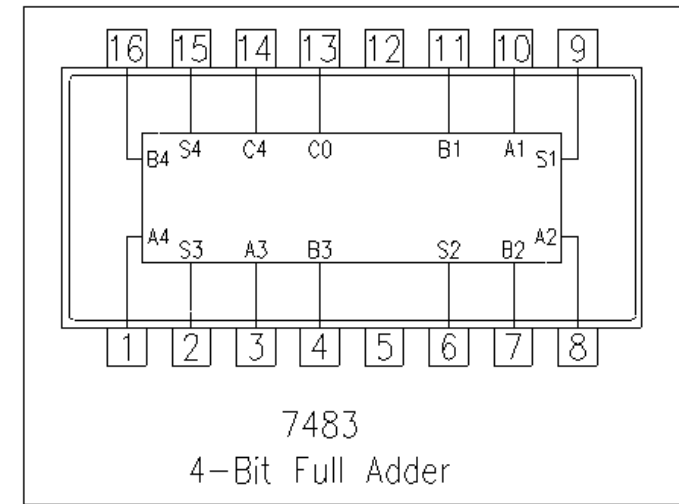
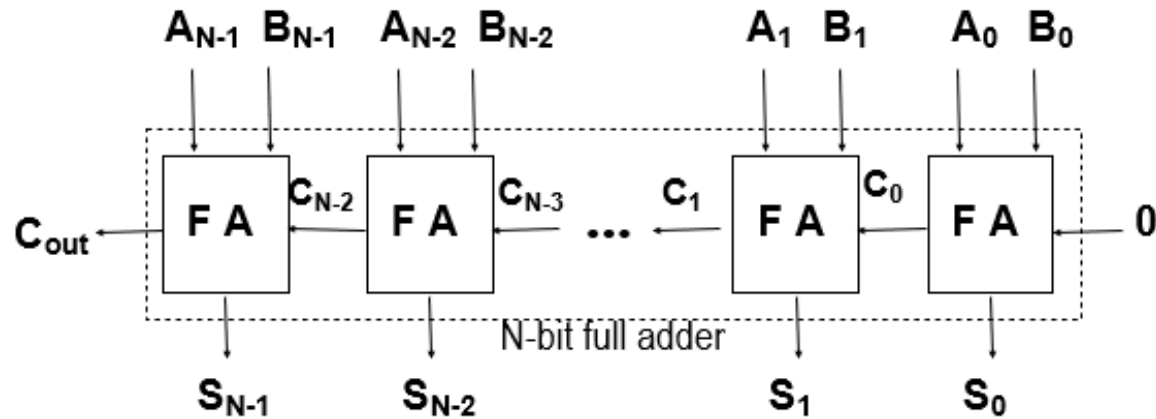


Inputs - A, B, C_{in} (each one bit)
Outputs - S, C_{out} (each one bit)

- Homework – Derive the circuit from the truth tables.

N-bit Full Adder

- Following the same logic, we can create an n-bit adder.



- This can conduct 4-bit (nibbles) operations such as $0100 + 0101$ which we saw in Week 1 & 2 lectures.

Subtraction/Negative Numbers

- Recall Two's compliment.
- $A - B = A + (-B)$
- Using two's compliment, $-B = \bar{B} + 1$

```
  0111  (A = 7)
+ 1011  (Two's complement of B = -5)
-----
 10010
```

- Since it's a 4bit system, the carry is discarded.

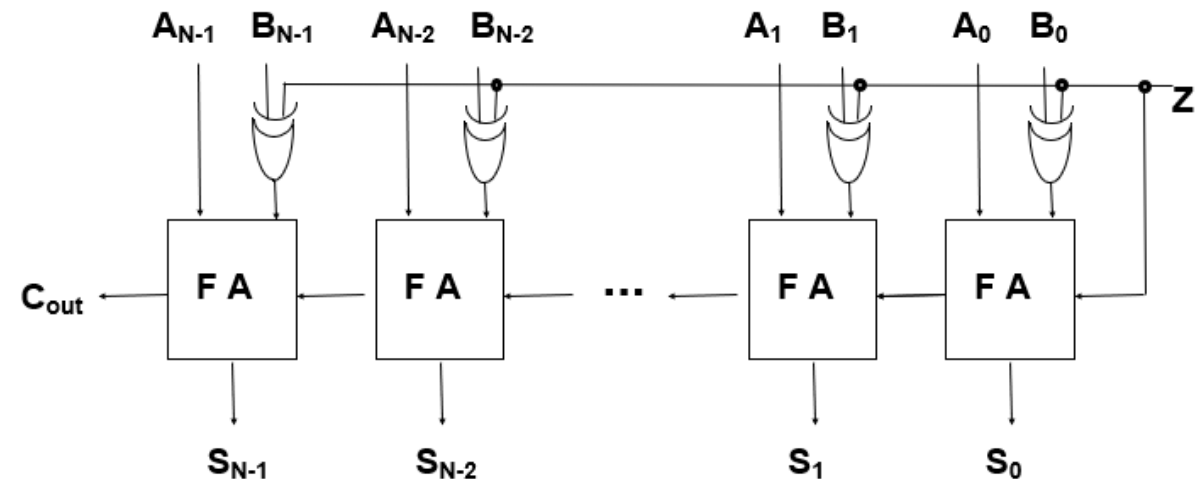
Adder $\rightarrow$ Adder/Subtractor

- To convert an adder into an adder/subtractor we simply add a control input Z such that:
- $Z = 0 \Rightarrow S = A + B$
- $Z = 1 \Rightarrow S = A - B$
- Remember that $A - B = A + (-B)$
- We can calculate $(-B)$ using two's complement
 - Invert the N-bit binary number B by finding $Z \oplus B$
 - Use Z as the Carry-In of the adder

Z	B	$Z \oplus B$	
0	0	0	} = B
0	1	1	
1	0	1	} = $\overline{B}$
1	1	0	

N-bit Adder/Subtractor

- A reasonably complex combinatorial circuit
- One of the more interesting circuits we've seen with respect to its function
- Try to see how this works for yourself!



Homework:

- What does the final carry indicate?

$Z = 0 \Rightarrow$ Adder

$Z = 1 \Rightarrow$ Subtractor

Active-High Vs Active-Low

- Going forward, we will see more control signals in circuits. So, its important to understand the difference between these 2 states.

- Active High

0 = inactive 1 = active



- Active Low

0 = active 1 = inactive

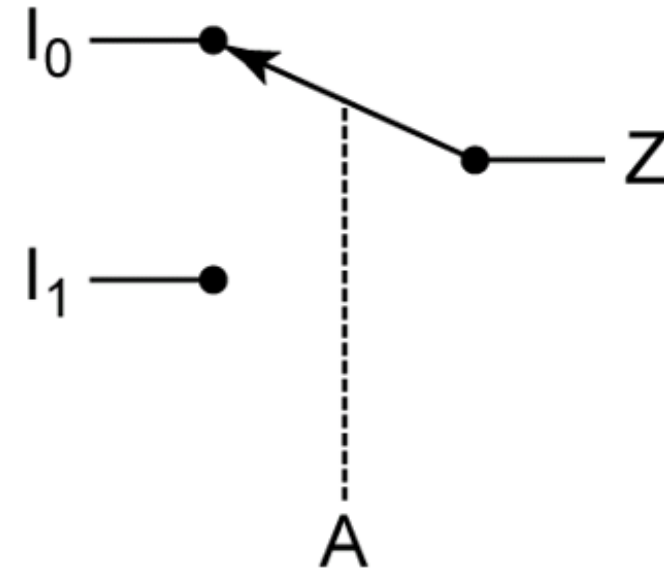
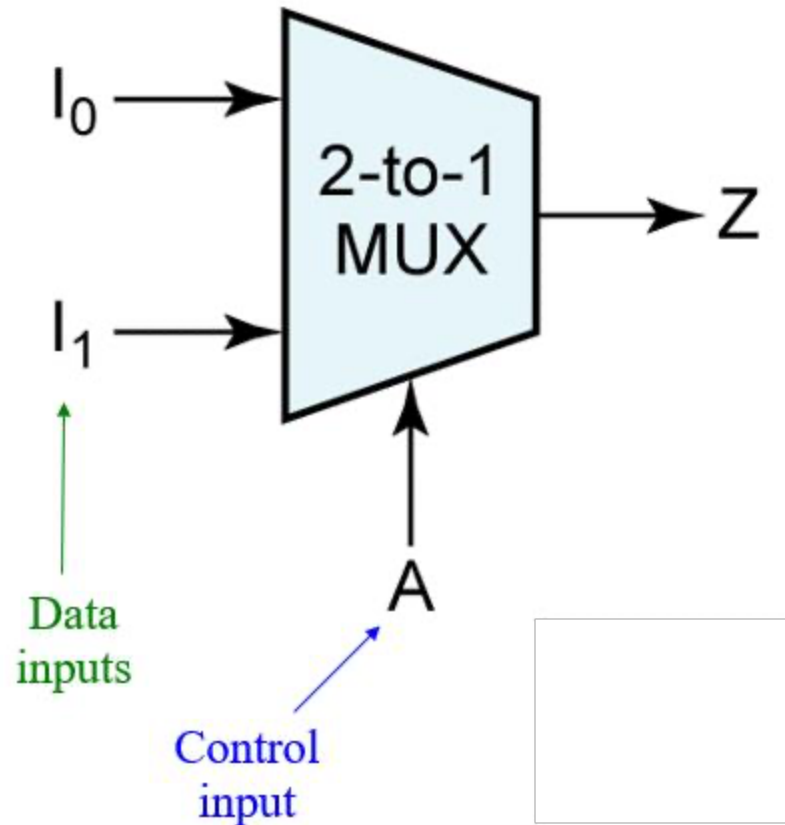


Pin sometimes labelled with a bar, e.g., $\overline{Enable}$

Multiplexers (MUX)

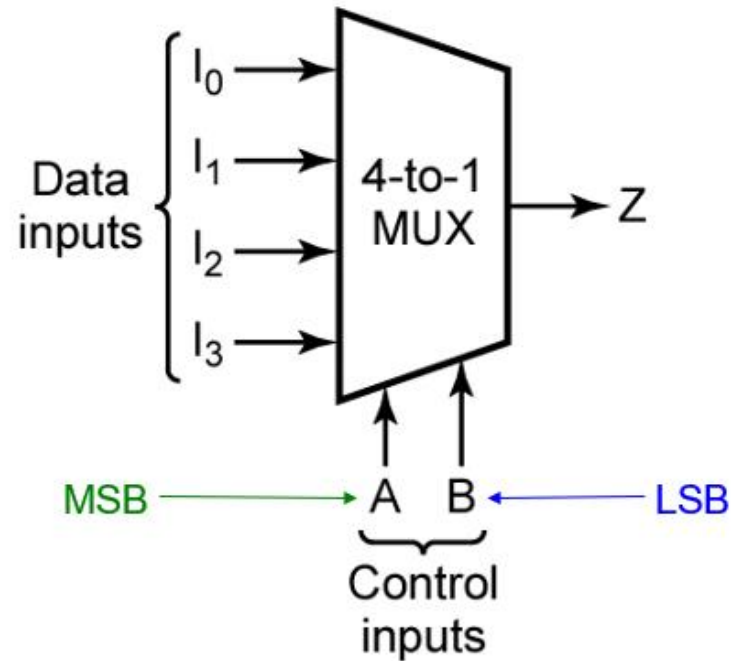
- Another very useful electronic component
- A MUX has,
 - N Control inputs
 - 2^N Data inputs
 - 1 Output
- A Multiplexer routes (connect) the selected data input to the output (A channel selector).
- The control input decides which data input is selected.

Multiplexers (MUX) Contd



$$Z = \bar{A}.I_0 + A.I_1$$

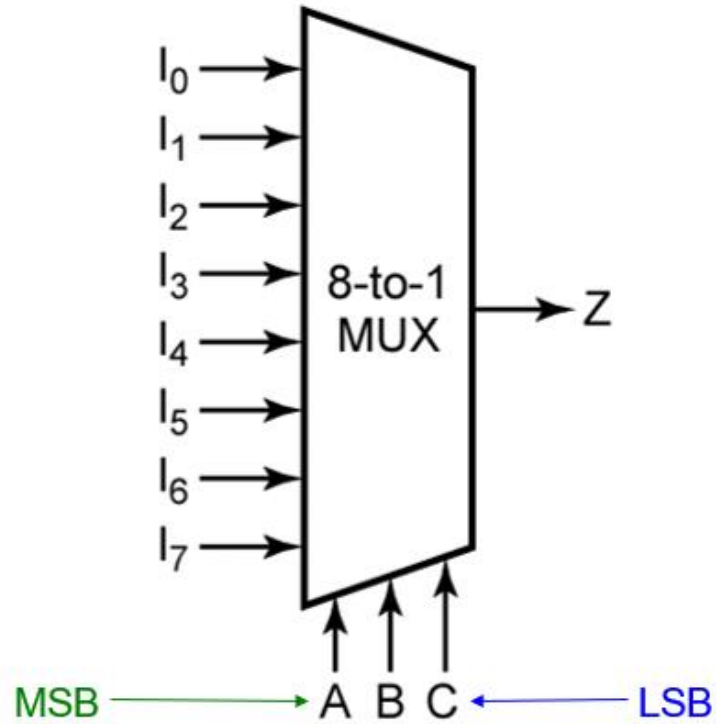
Multiplexers (MUX) Contd



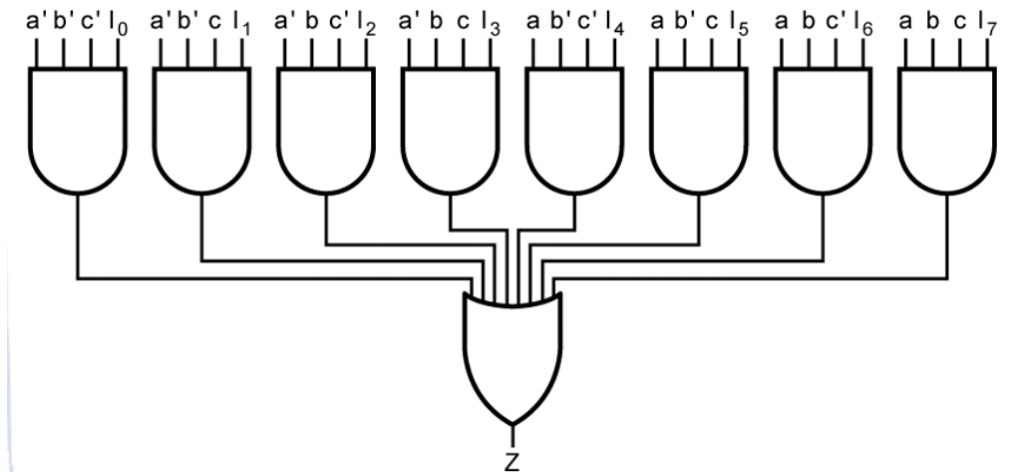
A	B	F
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Z = \bar{A}.\bar{B}.I_0 + \bar{A}.B.I_1 + A.\bar{B}.I_2 + A.B.I_3$$

Multiplexers (MUX) Contd



A	B	C	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7



Homework: Write the equation for Z

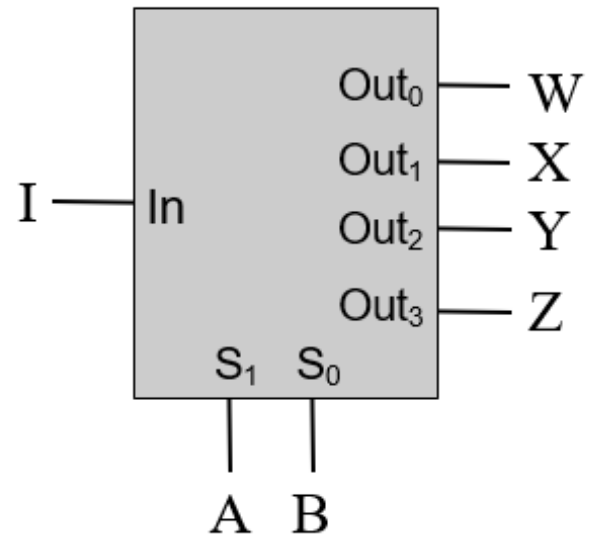
Common Applications of a MUX

- Source selection control
 - Home stereo, e.g., send iPod, CD or radio to speakers – note that this is analogue not digital
- Share one communication line between multiple senders
 - Requires both MUX and DE-MUX
- Parallel to serial conversion
 - Parallel input on X, clock signal on S, serial output on Y

Demultiplexers (DE-MUX)

- The opposite function than a Multiplexer...
- A DE-MUX has,
 - N Control Inputs
 - 1 Data Input
 - 2^N Outputs
- A demultiplexer routes (connect) the 1 data input to a selected output (signal distributor).

Demultiplexers (DE-MUX) Cont



A	B	W	X	Y	Z
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

$$W = I \bar{A} \bar{B}$$

$$X = I \bar{A} B$$

$$Y = I A \bar{B}$$

$$Z = I A B$$

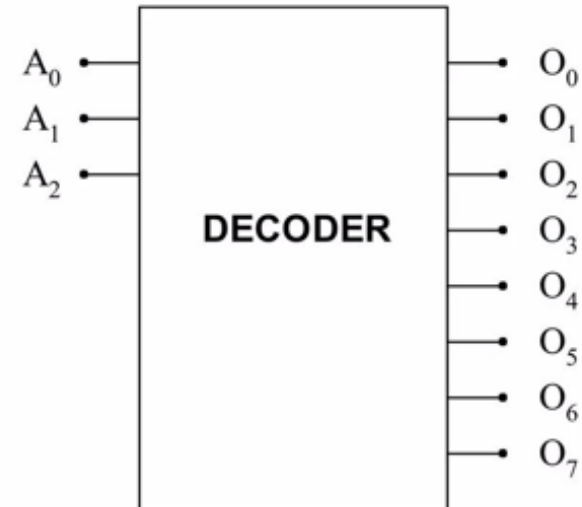
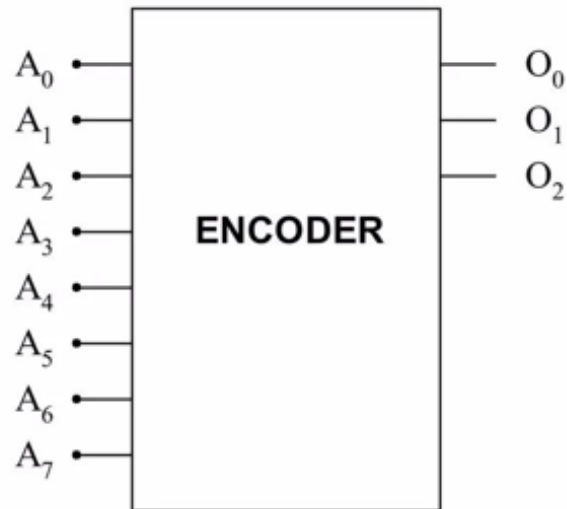
Common Applications of a DE-MUX

- Share one communication line between multiple senders
 - Requires both MUX and DE-MUX
- Serial to parallel conversion
- A control for multiple lights
 - In a gambling machine you might connect a processor to A and S and connect Y outputs to lights, such that the processor runs in a rapid loop addressing each light sequentially

Combinational Components: Decoders, Encoders & BCD Systems

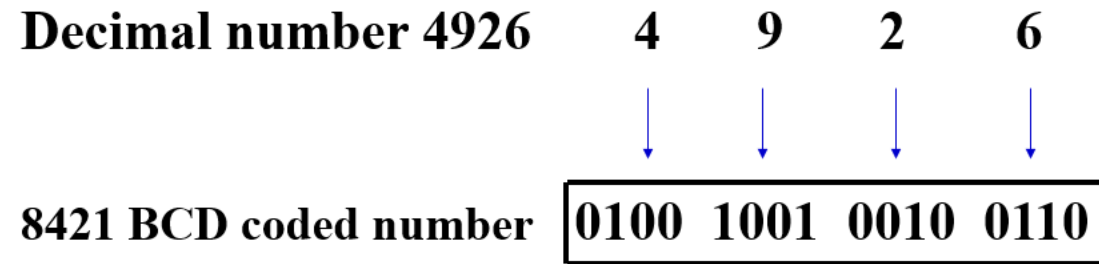
Encoders Vs Decoders

- Encoder: A circuit that produces a Binary input code depending on which of the inputs are activated.
- Decoder: A circuit that maps an N-bit code to 2^N one-hot outputs



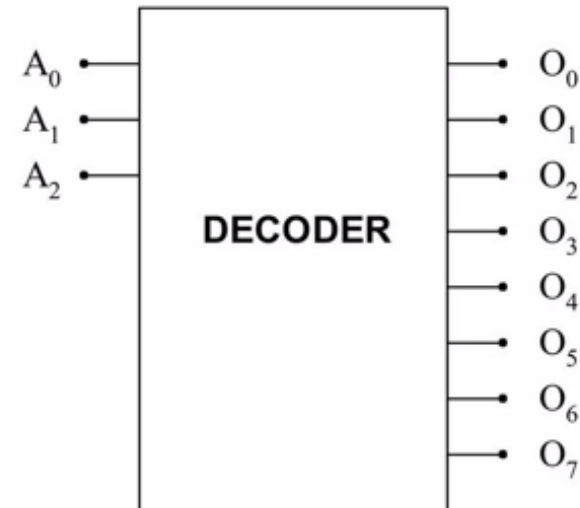
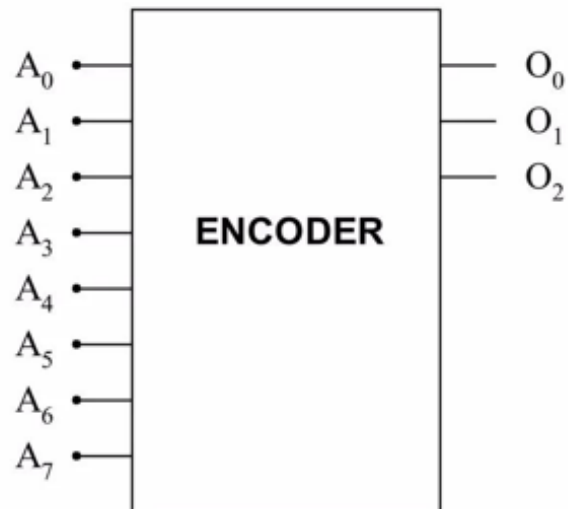
The 8421 BCD Code

- 8421 BCD (Binary Coded Decimal) is the most commonly used BCD format.
- It represents each decimal digit (0–9) using 4 binary bits.



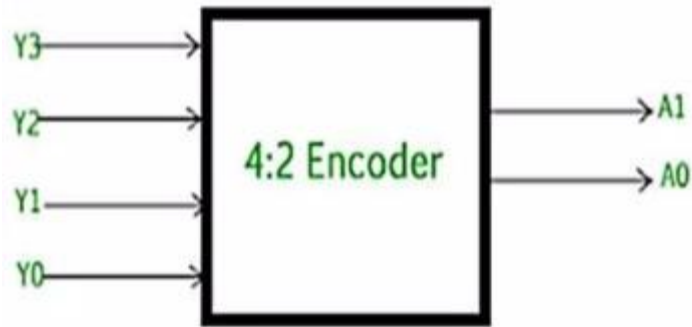
Encoders Vs Decoders Cont

Device	Role	Relation to BCD
Encoder	Converts decimal inputs (0–9) into 4-bit BCD code	This is where BCD encoding happens
Decoder	Converts 4-bit BCD input into one of 10 outputs (often driving display segments or indicators)	Used when reading or displaying BCD values



Encoders

- An encoder will have,
 - 2^N inputs
 - N outputs
- Let's consider a simple 4:2 binary encoder first,



I_3	I_2	I_1	I_0	Y_1	Y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

- Q. Is this “Active High” or “Active Low”?

Encoders Cont.

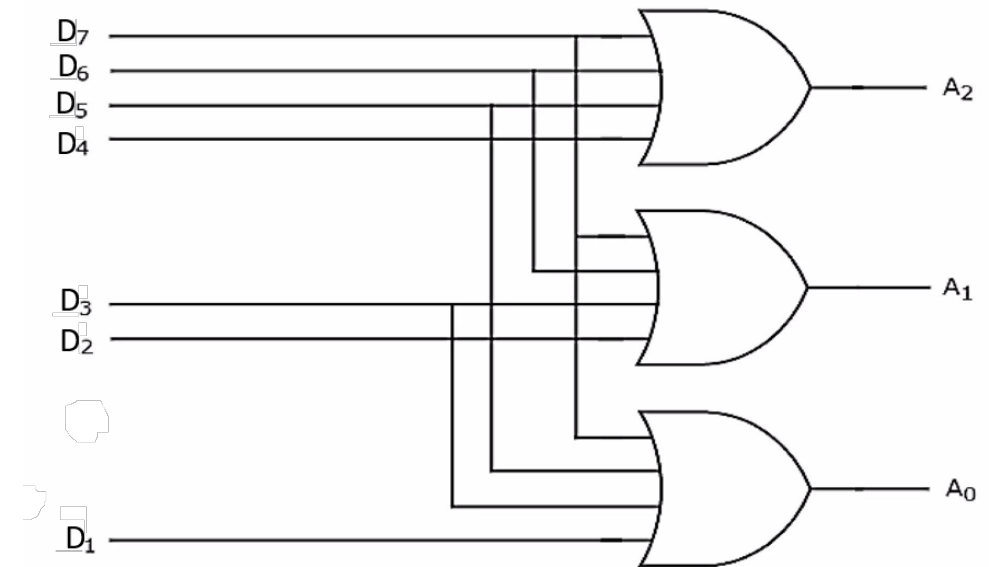
- Now let's consider a simple 8:3 binary encoder,

Inputs								Outputs		
D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀	A ₂	A ₁	A ₀
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A_0 = D_1 + D_3 + D_5 + D_7$$

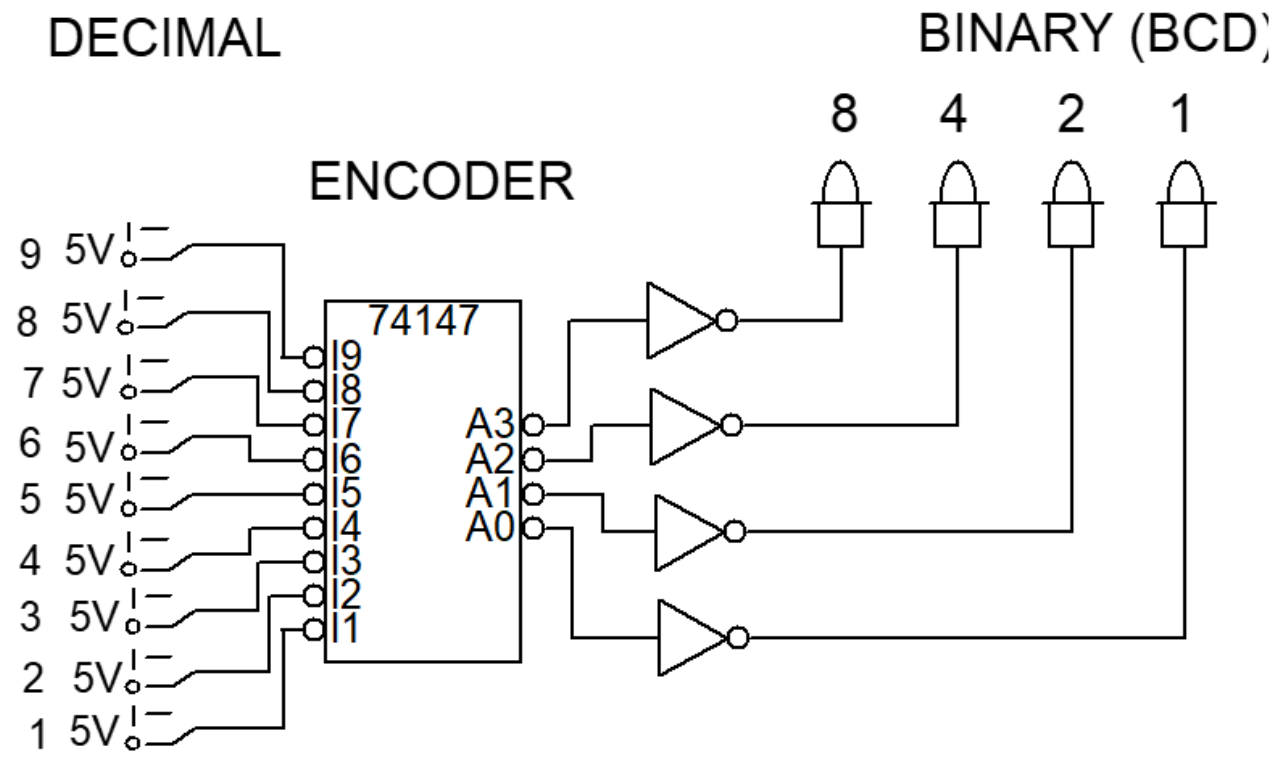
$$A_1 = D_2 + D_3 + D_6 + D_7$$

$$A_2 = D_4 + D_5 + D_6 + D_7$$



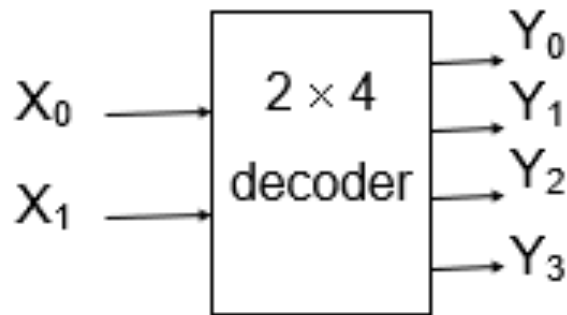
Encoders Cont.

- Circuit diagram of a 10:4 BCD encoder:



Decoders

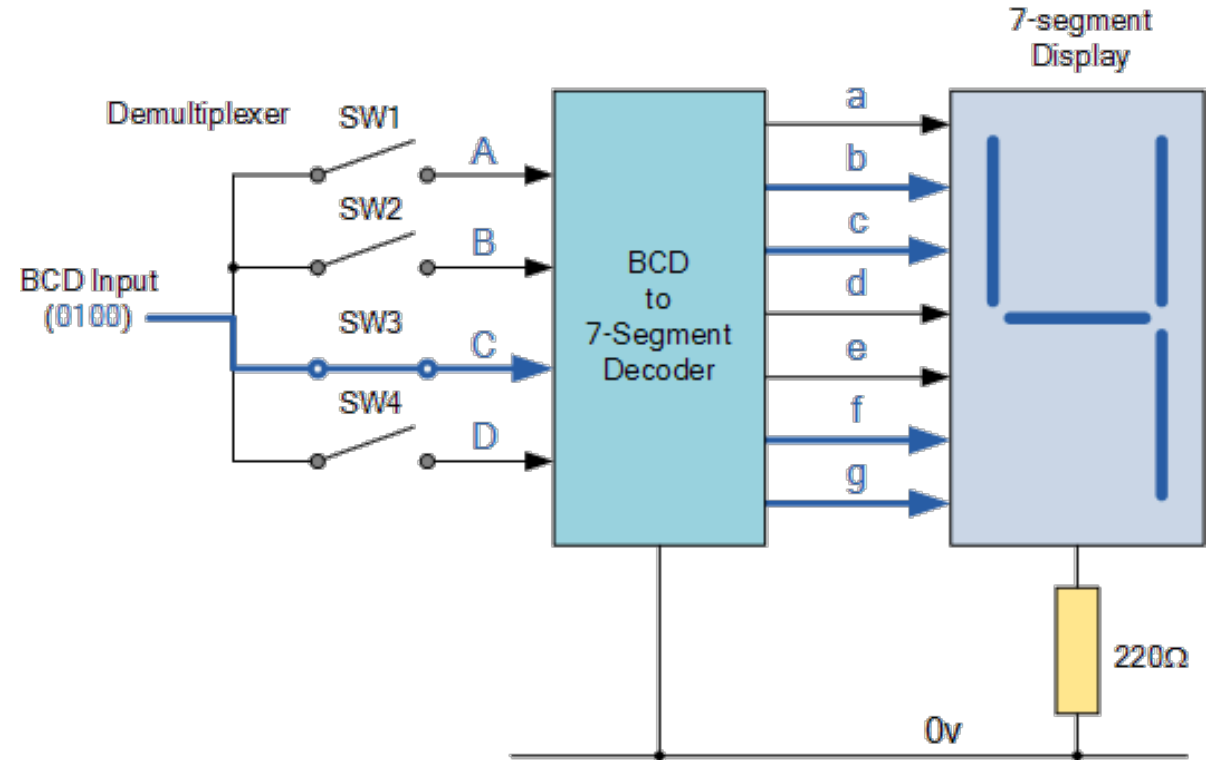
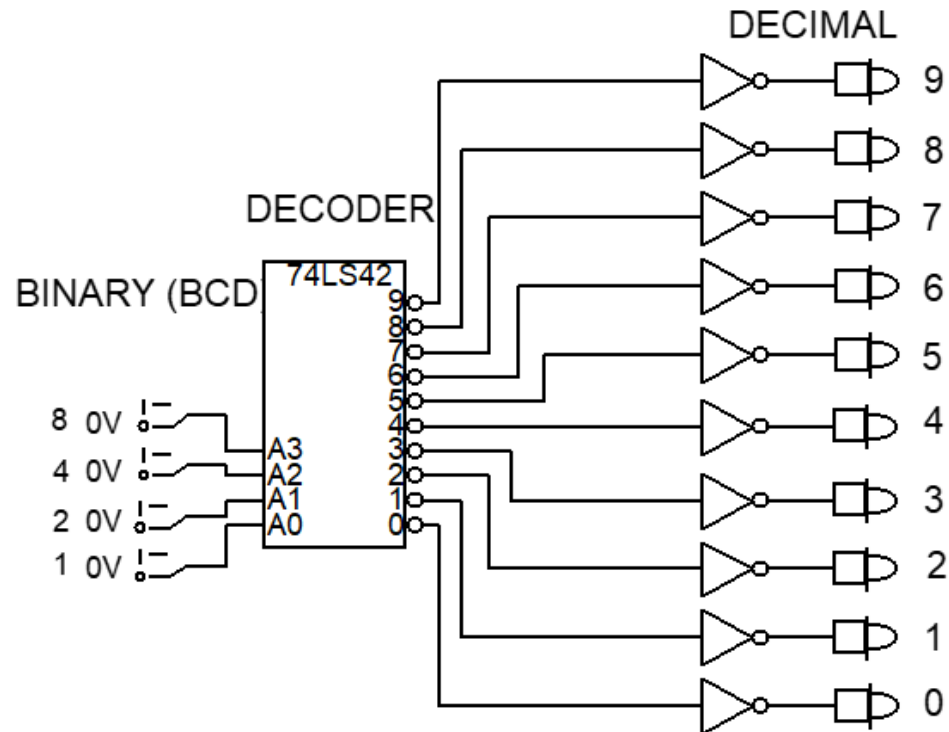
- A decoder will have,
 - N inputs
 - 2^N outputs
- The exact opposite function of an encoder...



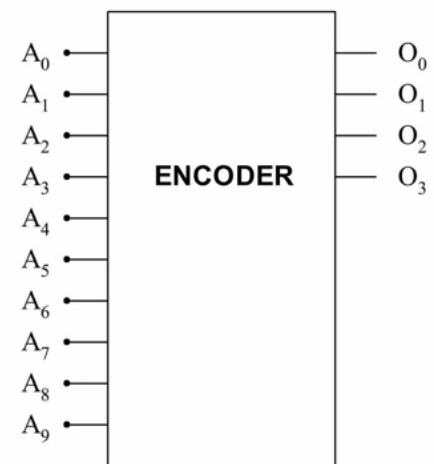
X_0	X_1	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- Extra work: Work out the Boolean logic expressions for Active High and Active Low decoders....
- Bonus – Implement with NAND gates.

Decoders Cont.

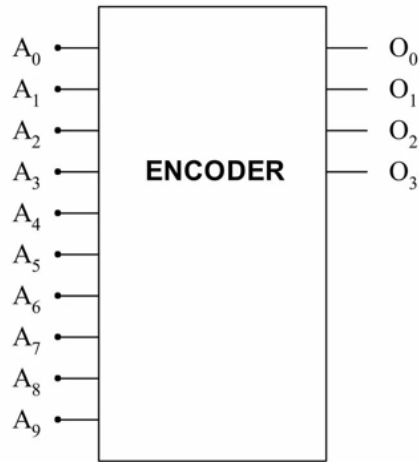


Try filling this table.



Input	O_3	O_2	O_1	O_0
A1=1				
A4=1				
A6=1				
A8=1				

Try filling this table.

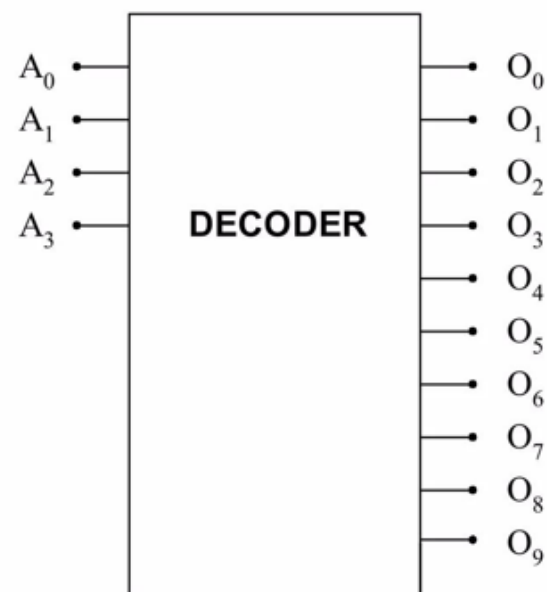


Input	O_3	O_2	O_1	O_0
A1=1	0	0	0	1
A4=1	0	1	0	0
A6=1	0	1	1	0
A8=1	1	0	0	0

Bonus Q:

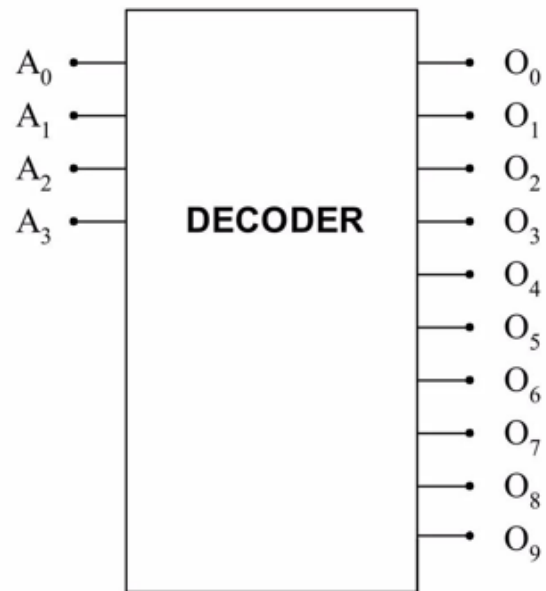
1. What if there is more than 1 input selected at a time? How can we prevent this?
2. What if none of the inputs are selected at a given time? How can we fix this?

Try filling this table.



A_3	A_2	A_1	A_0	Output
0	0	0	0	
0	1	0	1	
0	1	1	1	
1	0	0	1	

Try filling this table.



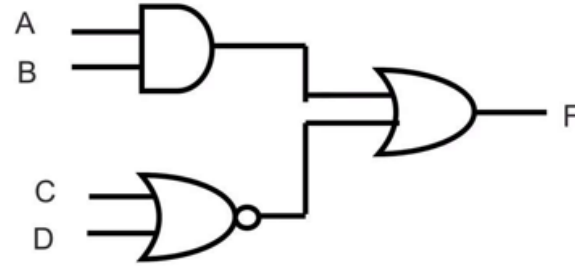
A_3	A_2	A_1	A_0	Output
0	0	0	0	$O_0=1$
0	1	0	1	$O_5=1$
0	1	1	1	$O_7=1$
1	0	0	1	$O_9=1$

Sequential Logic Circuits

Sequential Vs Combinational

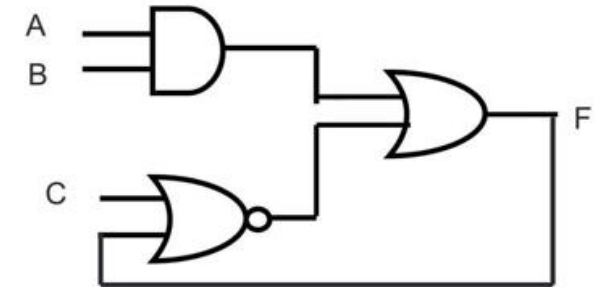
Combinational Circuit

- output determined solely by inputs

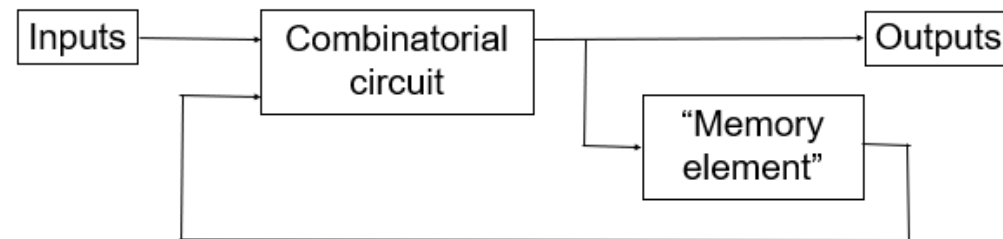


Sequential Circuit

- output determined by inputs AND previous outputs

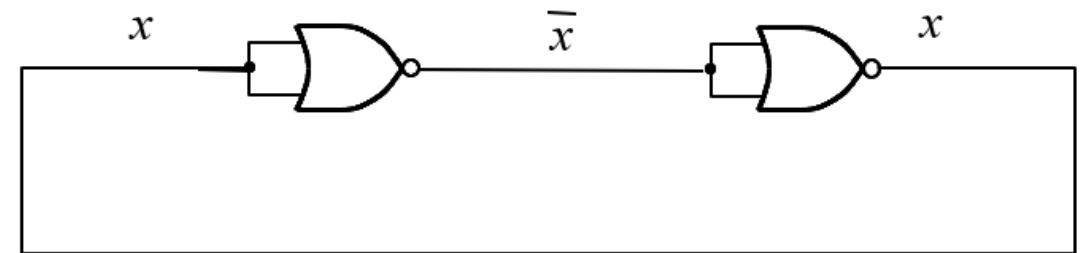
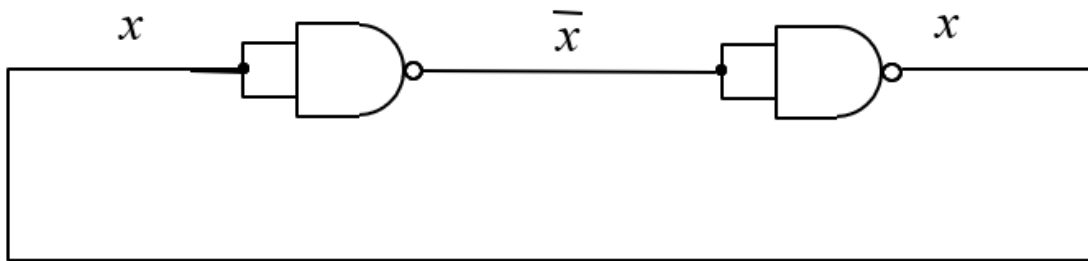
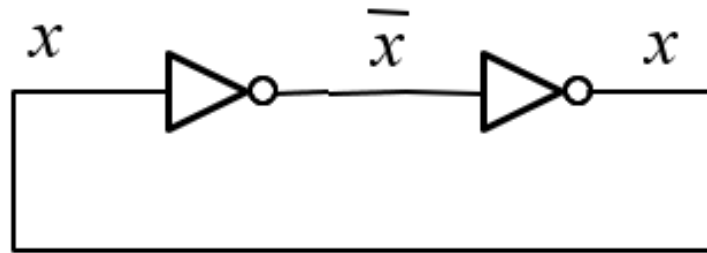


- Now we are getting into circuits which has a Memory component involved.

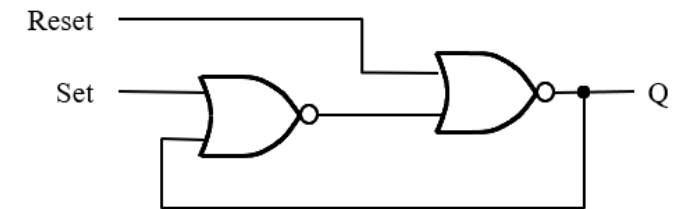
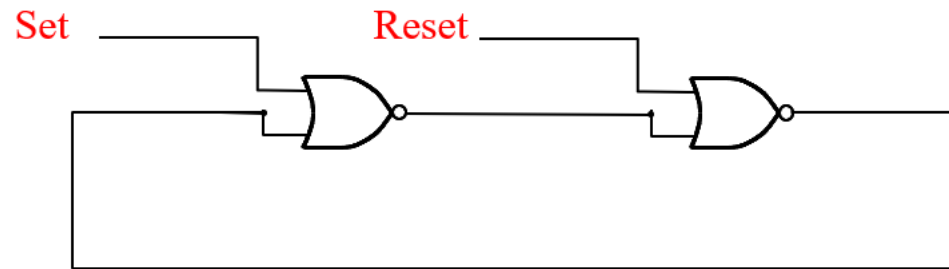
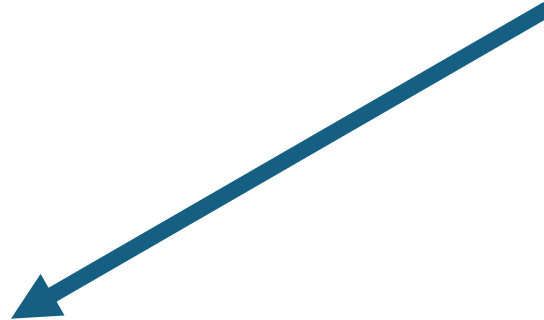
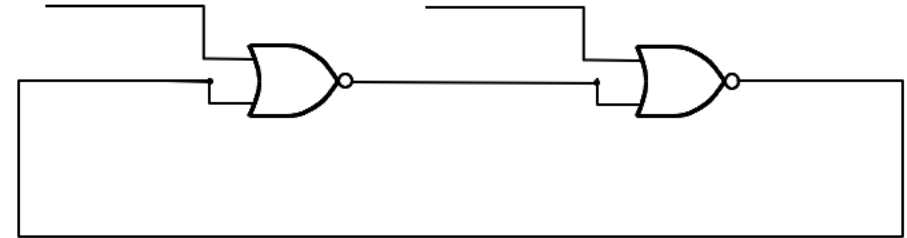
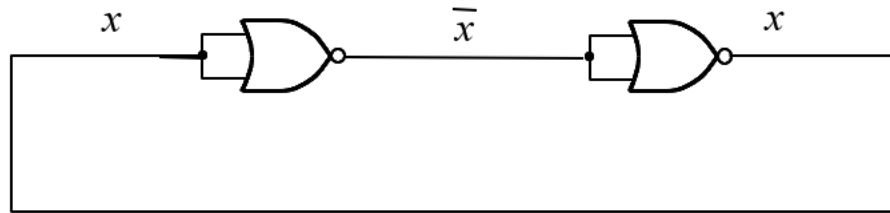


Simple Memory Elements (for intuition)

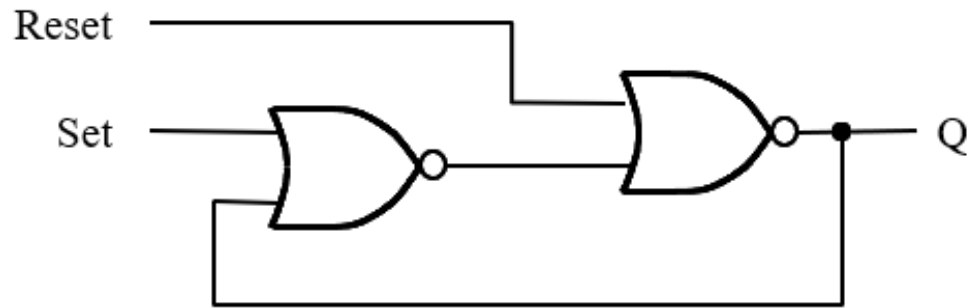
- Memory involves in retention (holding on) the current value.



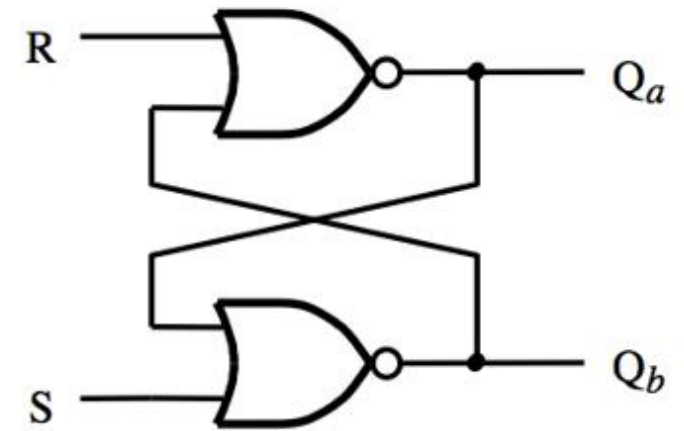
Basic Latch



Basic Latch Cont.



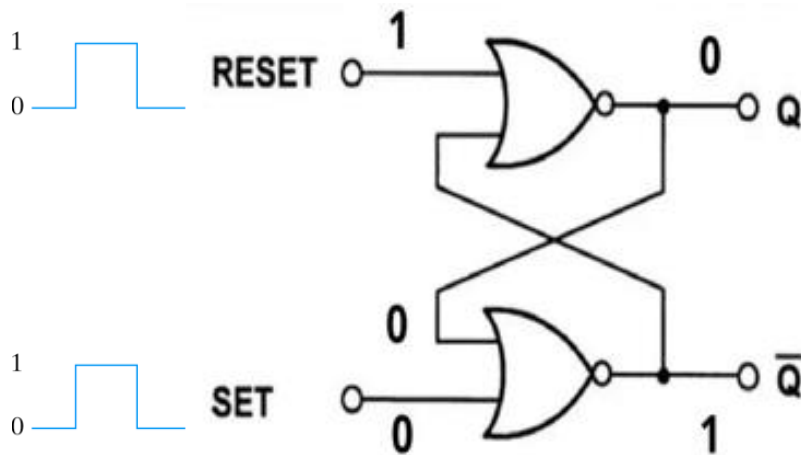
=



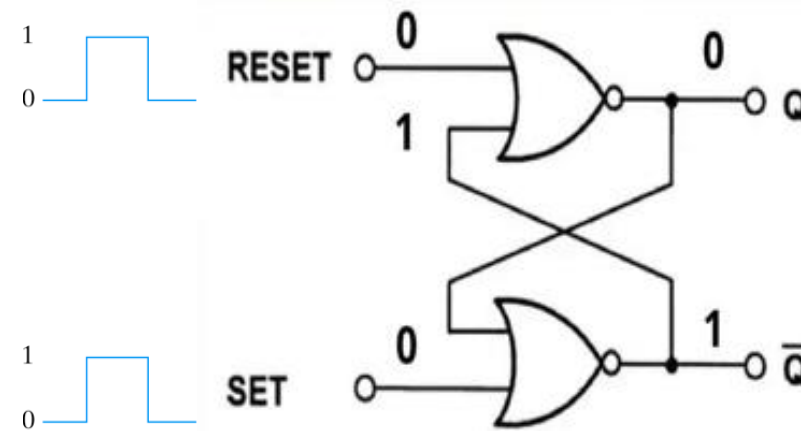
SR (Set-Reset) Latch

SR Latch

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0



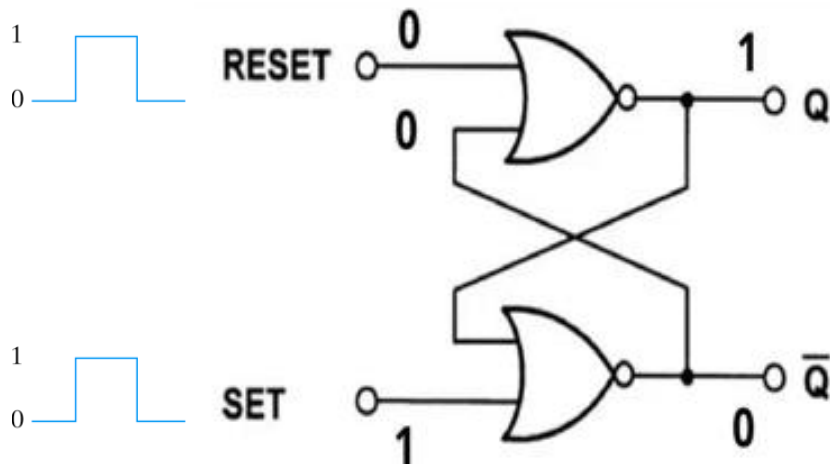
If $R=1$ & $S=0$,
Then $Q = 0$ and $\bar{Q} = 1$
Latch is **RESET**



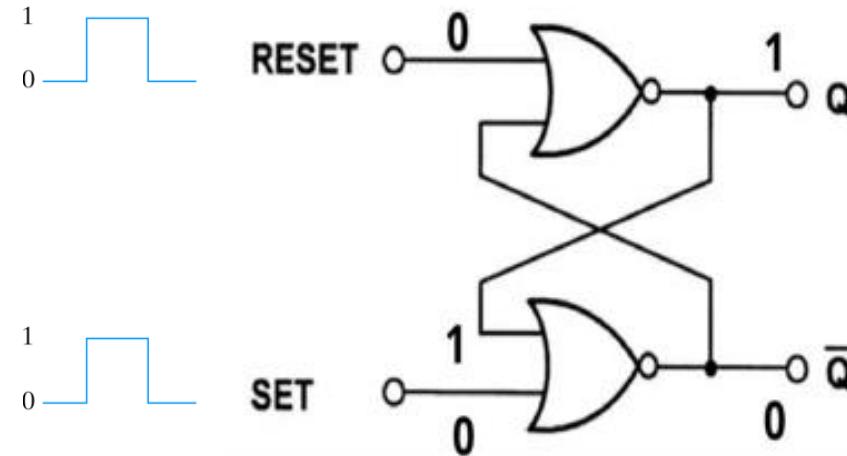
If $R=0$ & $S=0$,
Then $Q = 0$ and $\bar{Q} = 1$
No Change (Held in Memory)

SR Latch Cont

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

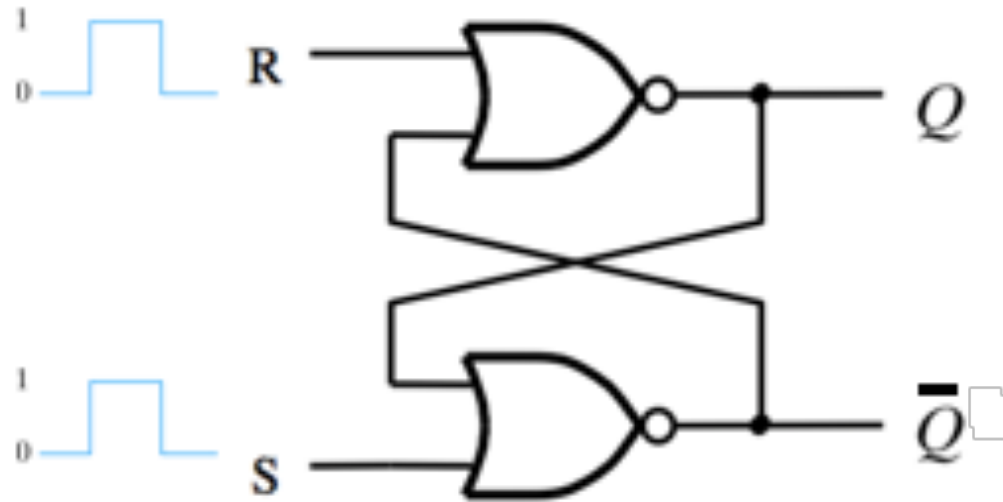


If $R=0$ & $S=1$,
Then $Q = 1$ and $\bar{Q} = 0$
Latch is **SET**



If $R=0$ & $S=0$,
Then $Q = 1$ and $\bar{Q} = 0$
No Change (Held in Memory)

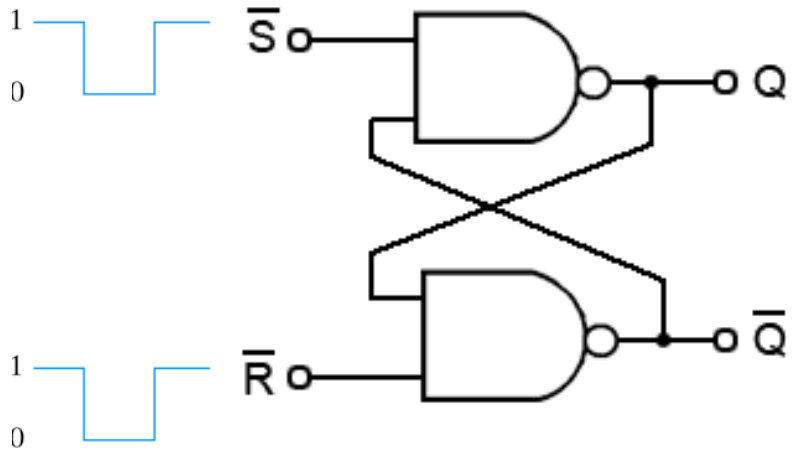
SR Latch – Active High (NOR)



S	R	Q	$\bar{Q}$	
0	0	0/1	1/0	(no change)
0	1	0	1	
1	0	1	0	
1	1	0	0	← UNDEFINED STATE

- In the last case when both S and R are “1”, we have an undefined state, where the latch breaks its condition: Q and $\bar{Q}$ should **always be exact opposites**.
- Therefore, we need to take care to AVOID this situation at all cost.

SR Latch – Active Low (NAND)



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	X	X
0	1	1	0
1	0	0	1
1	1	no change	

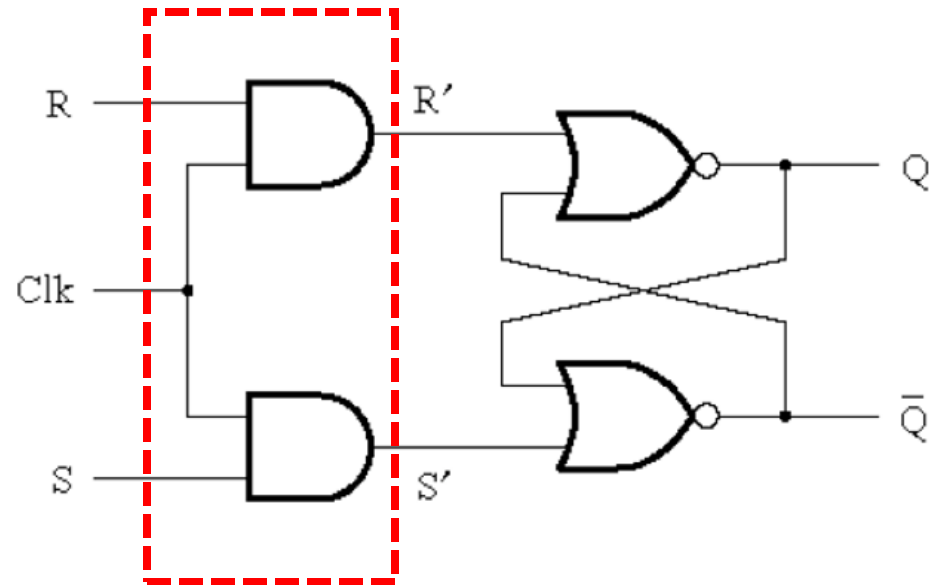
← UNDEFINED STATE

- In the last case when both inputs are “0”, we have an undefined state, where the latch breaks its condition: Q and $\bar{Q}$ should **always be exact opposites**.
- Therefore, we need to take care to AVOID this state...
- Existence of such states to avoid is Extra Work for designers.

Gated SR Latch

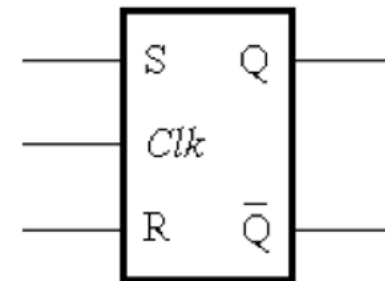
- A basic latch changes its output immediately whenever the inputs change — this is called asynchronous behavior.
- This can be unpredictable, because input signals may change at random times, making it hard to control when the latch updates.
- To gain control, we introduce an Enable (or Gate) signal.
- When Enable = active, the latch is transparent → it follows the inputs.
- When Enable = inactive, the latch ignores inputs and holds its previous value.
- If the Enable signal is driven by a Clock, the latch becomes a Clocked (Gated) Latch, meaning it only updates during the clock's active level.

Gated SR Latch

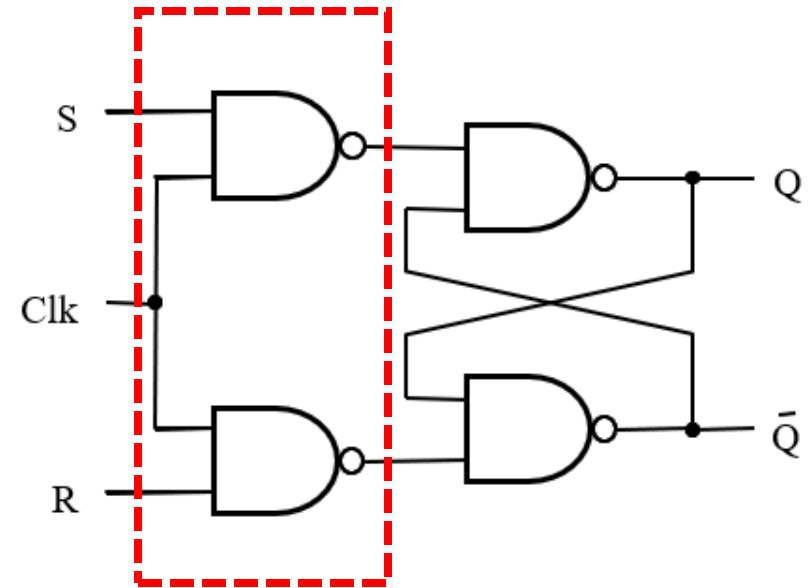
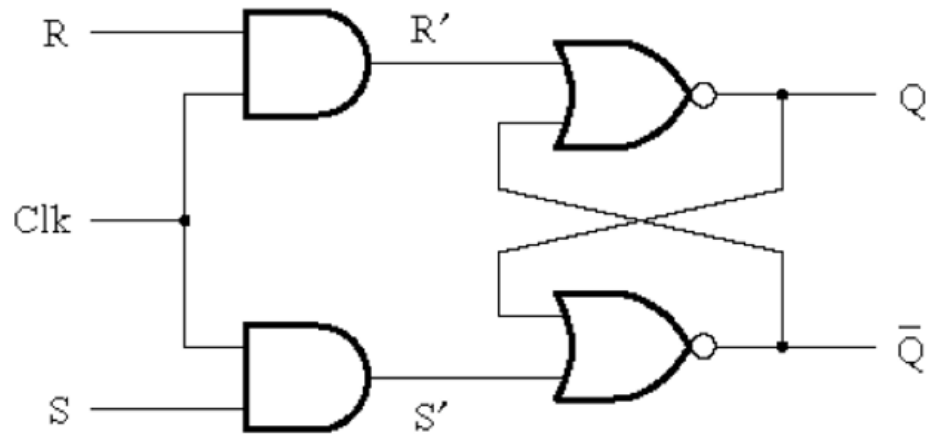


This is the
“GATE” in the
Gated Latch.

Clk	S	R	$Q(t+1)$
0	x	x	$Q(t)$ (no change)
1	0	0	$Q(t)$ (no change)
1	0	1	0
1	1	0	1
1	1	1	x



Gated SR Latch Variations

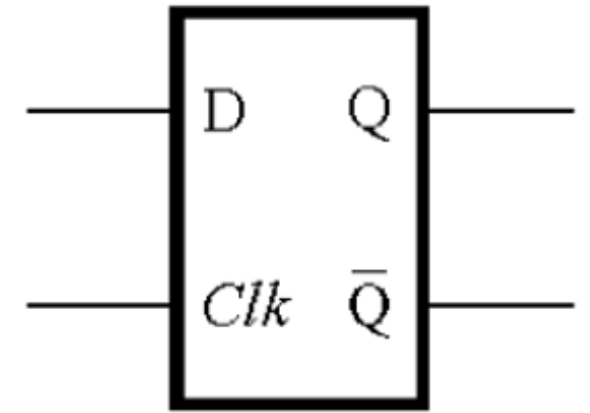
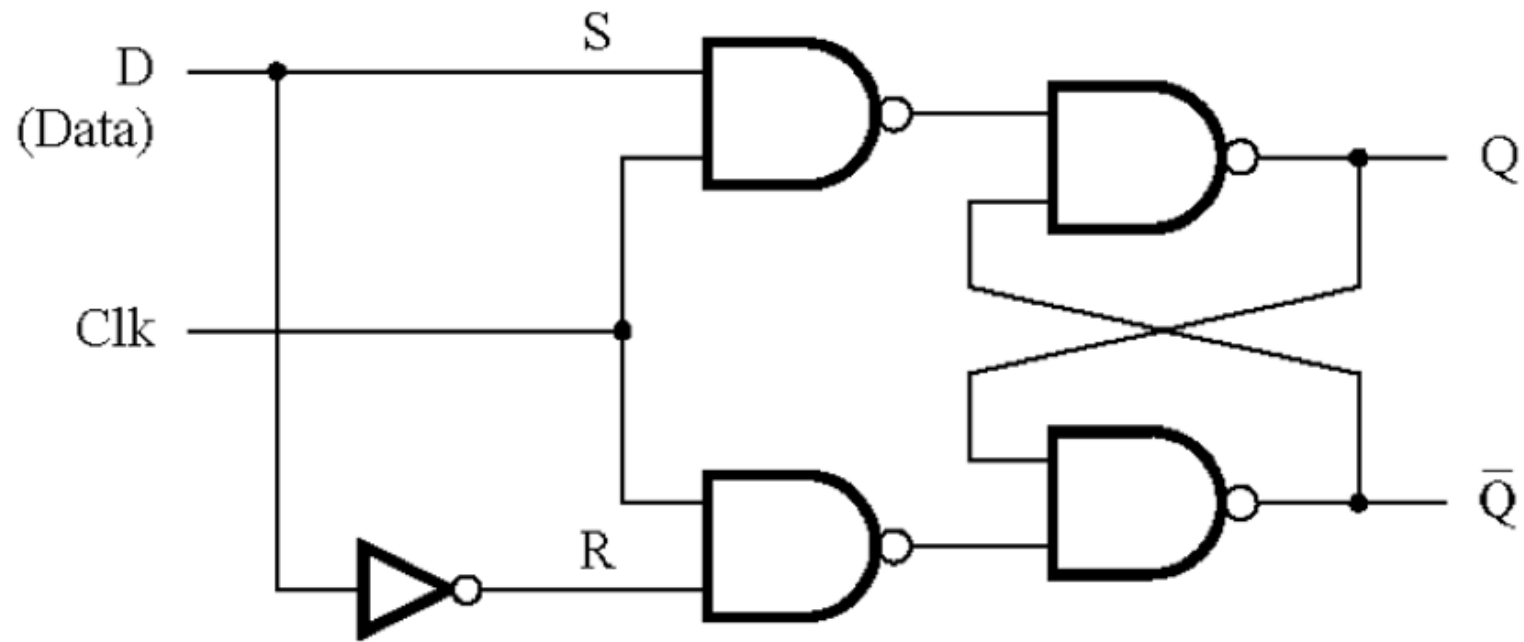


Note that its Gated
with NAND Gates
(Not AND Gates)

Gated D Latch

- Using two inputs (S and R) can be inconvenient.
- For example, to store a 1, we must Set ($S=1$) — but only if it was not already 1.
- If the previous state is unknown, we might unnecessarily Reset first, which complicates control.
- Why not use a single input that directly represents the desired value?
- This leads to the D (Data or Delay) Latch, which is simply a modified SR latch where:
 - $D = 1 \rightarrow \text{Set}$,
 - $D = 0 \rightarrow \text{Reset}$ (automatically handled by an inverter)

Gated D Latch



FlipFlops

- Are always Clocked.
- Constructed from Latches from along with additional clock signals.
- Since flip-flops wait for a clock edge, they respond less immediately than level-sensitive latches.

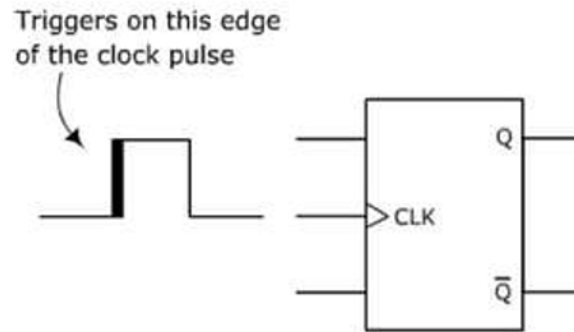
Type	Meaning	Example	When Does It Change Output?
Asynchronous Storage Element	Responds immediately to input changes (no clock involved)	SR Latch, D Latch	Output changes as soon as S or R (or input) changes
Synchronous Storage Element	Responds only when a CLOCK signal allows it	Flip-Flops (D FF, JK FF, T FF)	Output changes only on a specific clock edge (rising/falling)

Clocks

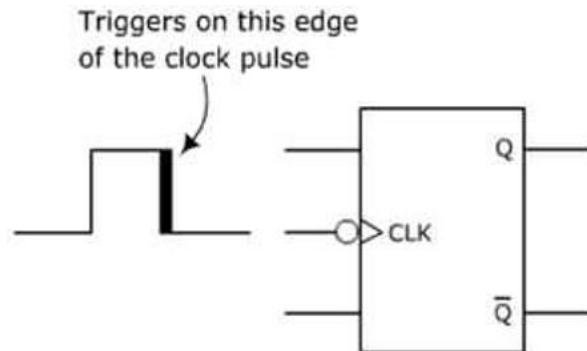
- A clock is a signal that repeatedly alternates between 0 and 1 at a fixed rate.
- The clock period (or cycle time) is the time for one complete cycle e.g., from one rising edge to the next rising edge.
- The clock frequency is $1 \div \text{clock period}$, measured in Hertz (Hz).
- Digital systems use clock signals to synchronize operations.

Triggering in Sequential Circuits

- Flip-flops (and many sequential circuits) change their output only when the clock allows it.
- The way a flip-flop responds to a clock is called clock triggering, and it can happen in two (four if you include Latches) ways:
- Positive-edge triggering ($\uparrow$) $\rightarrow$ Responds only when clock changes from 0 $\rightarrow$ 1



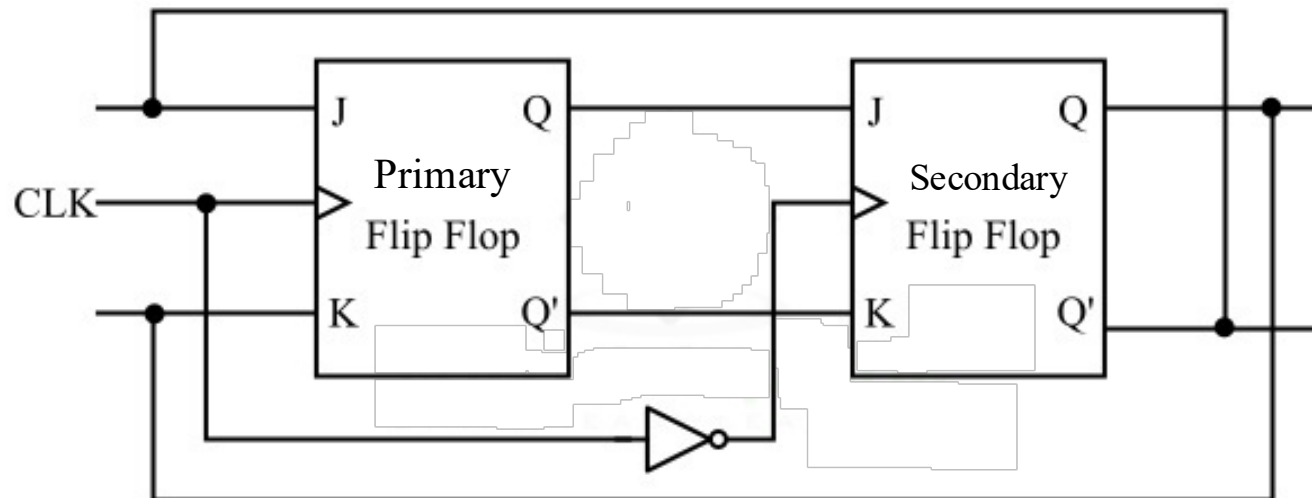
- Negative-edge triggering ($\downarrow$) $\rightarrow$ Responds only when clock changes from 1 $\rightarrow$ 0



JK Type Flipflops

- “J” and “K” are simply historical labels, named after the inventor **Jack Kilby** — they don’t stand for anything specific.
- Combines the behavior of SR, D, and T Flip-Flops into one universal device.
- If $J = 1$ and $K = 0 \rightarrow$ Set ($Q = 1$)
- If $J = 0$ and $K = 1 \rightarrow$ Reset ($Q = 0$)
- If $J = 0$ and $K = 0 \rightarrow$ Hold previous state
- If $J = 1$ and $K = 1 \rightarrow$ Toggle the output (covered soon)
- Free from the “invalid” state of the SR Flip-Flop — that’s why it’s considered a more robust version of SR.
- Often used in counters, control circuits, and sequential logic systems.

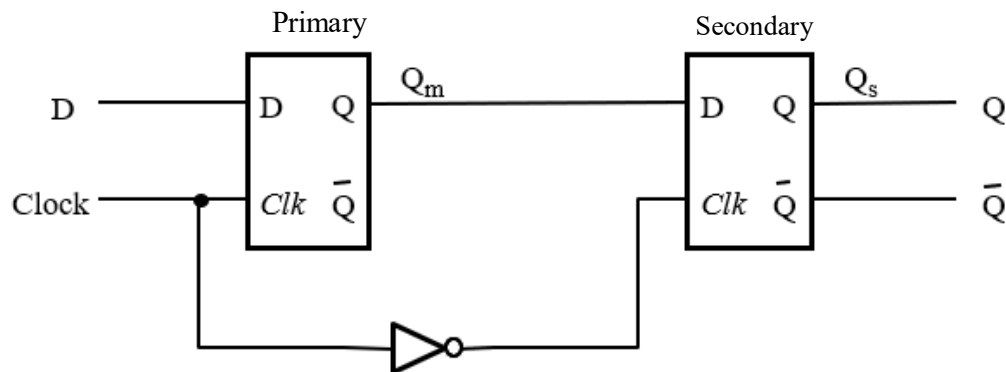
JK Type Flipflops



J	K	Clk	Q
0	0	↑	Q No change
0	1	↑	0
1	0	↑	1
1	1	↑	$\overline{Q}$

D Type Flipflops

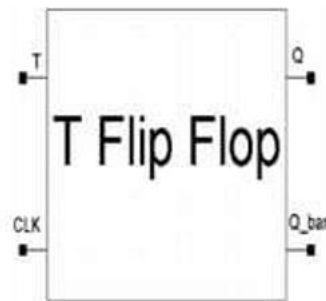
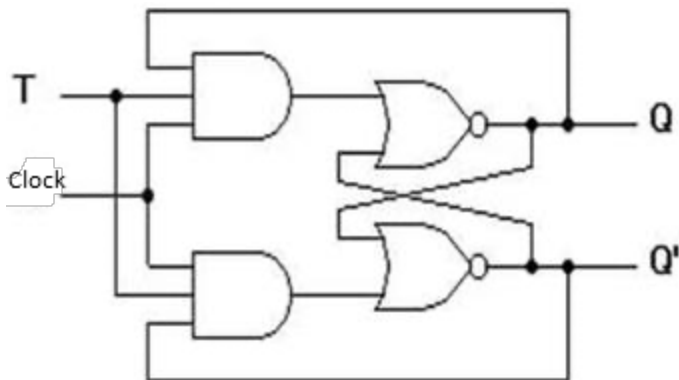
- D Flip-Flop = Special case of JK Flip-Flop when:
 $J = D$ and $K = \bar{D}$
- “D” stands for “Data”, and “Delay” (since Q is a delayed version of D)
- Works like a Gated D-Latch, but responds only at clock edges (edge-triggered, not level-triggered)
- Internally built using two latches in series (Primary-Secondary configuration)
→ No need to go into detail for this module



D _n	Clk	Q
0	↑	0
1	↑	1

T Type Flipflops

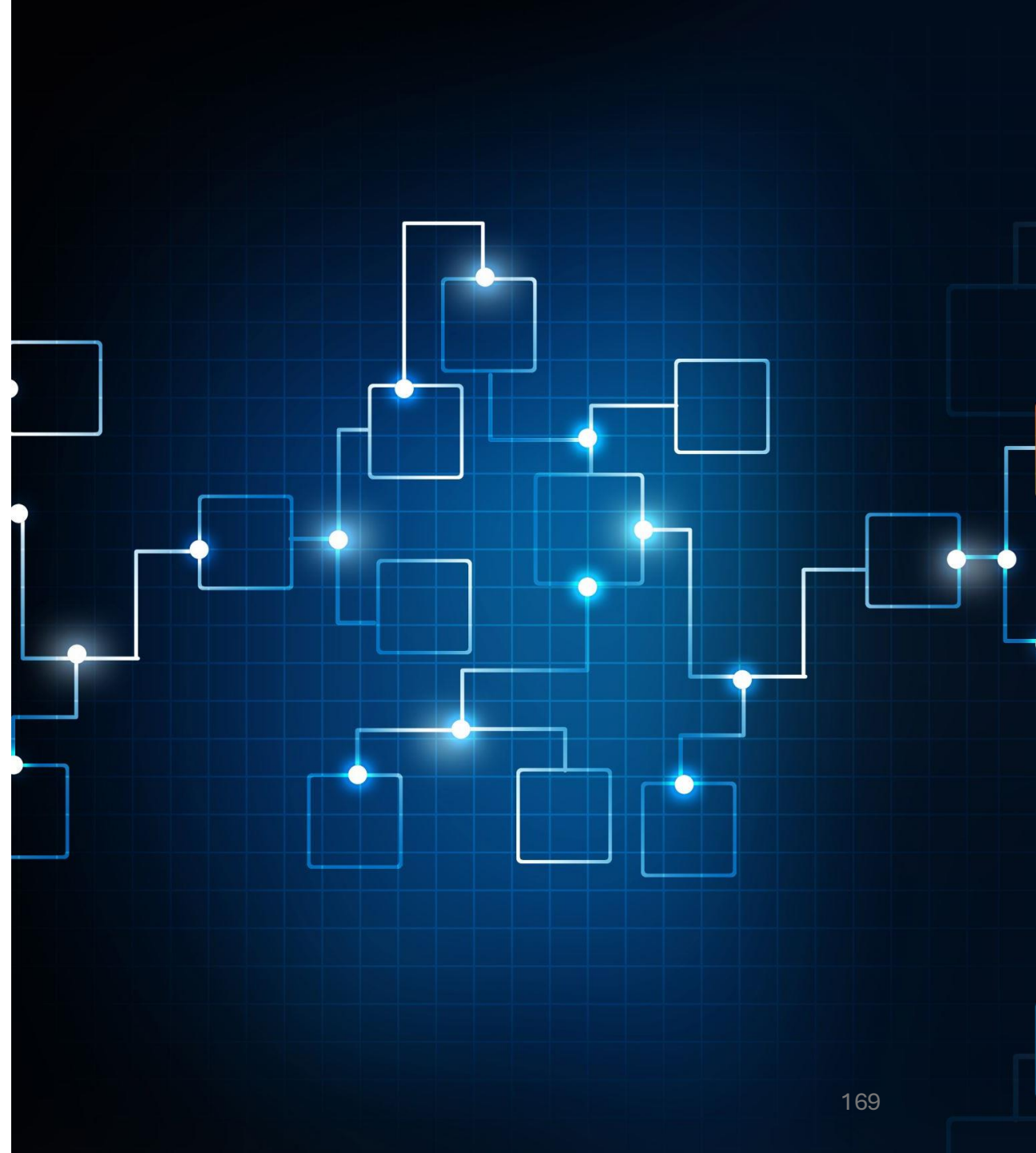
- “T” stands for “Toggle” — the output switches (flips) between 0 and 1 on every active clock edge when $T = 1$.
- Can be implemented using a JK Flip-Flop with $J = K$
- If $T = 0 \rightarrow$ Output holds previous state (no change).
- If $T = 1 \rightarrow$ Output toggles on each clock edge (edge-triggered, not level-triggered).
- Commonly used in binary counters, frequency dividers, and digital clocks.



T	Clk	Q	
0	↑	Q	Hold
1	↑	$\overline{Q}$	Toggle

Recap

- Combinational Vs Sequential circuits
- Half Adder → Full Adder (2 Half Adders + OR)
- Subtraction via Two's Complement
- Adder/Subtractor circuit using XOR + Carry-in
- Transition from Combinational → Sequential Logic
- Introduction to SR / D / JK / T Flip-Flops
- Clock-controlled state-holding circuits



Thank you....

Akila Subasinghe
a.m.subasinghe@bham.ac.uk